# COMP0222 Simultaneous Localisation and Mapping

## Lecture 01: Basic Concepts and Notation

**Simon Julier**

s.julier@ucl.ac.uk

**Sajad Saeedi**

s.saeedi@ucl.ac.uk

# Structure

- Localization, Mapping, Odometry, SLAM
- Sparse Feature Point Maps
- Notation

# Localization, Mapping, Odometry, SLAM

- **Localization, Mapping, Odometry, SLAM**
- ***Sparse Feature Point Maps***
- ***Notation***

# The Platform and the Map

Platform

Map

# Localization

**The platform uses the map to localise**

# Localization in Cities Using Street View

From "MAV Urban Localization from Google Street View Data"

# Mapping

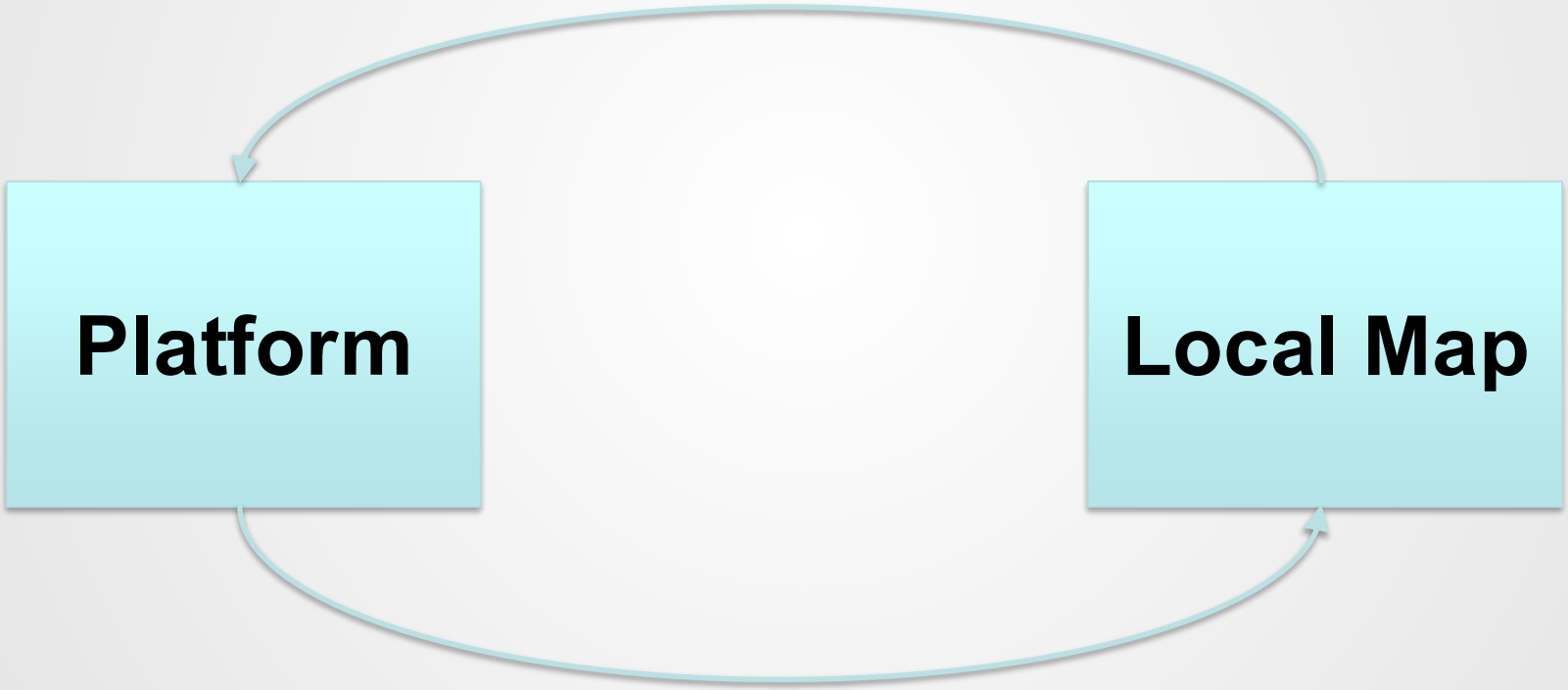
**The platform position is used to build the map**

# High Precision Mobile Scanning

Leica Absolute Tracker AT60, Leica T-Scan and use case from Hexagon MI

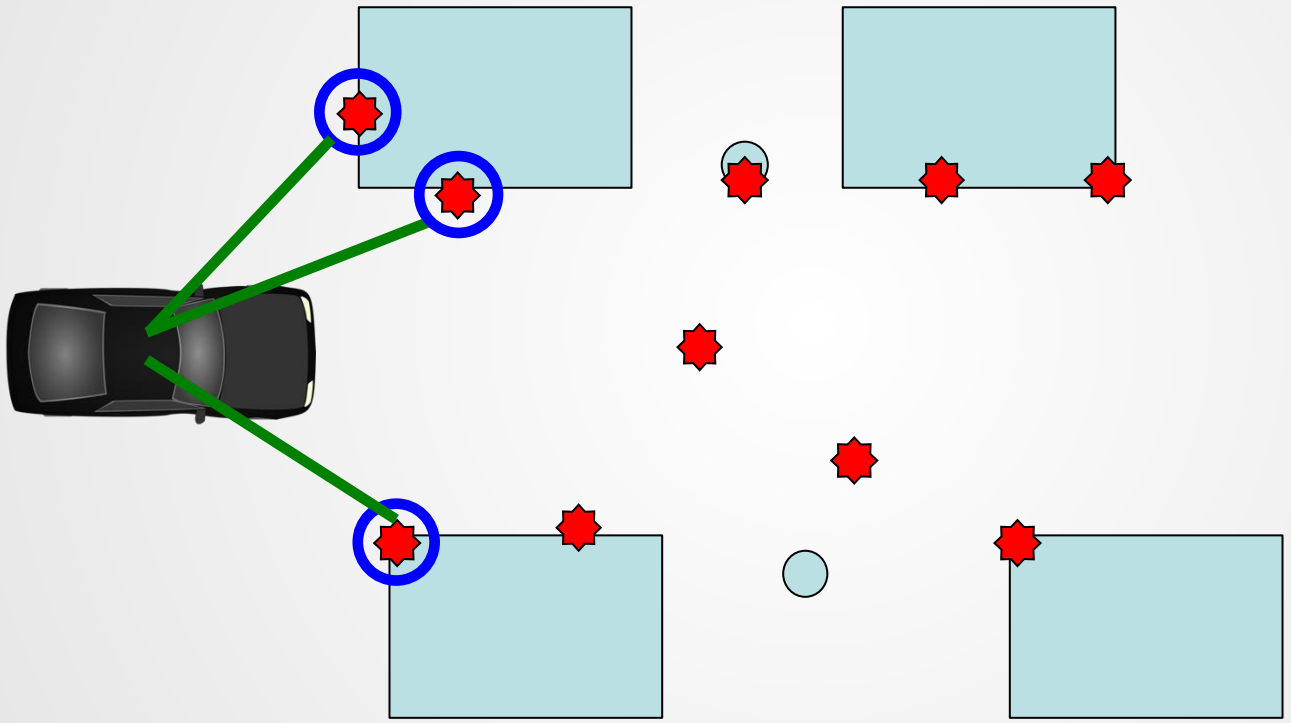
# Visual Odometry

**The platform uses the *local* map to localise**

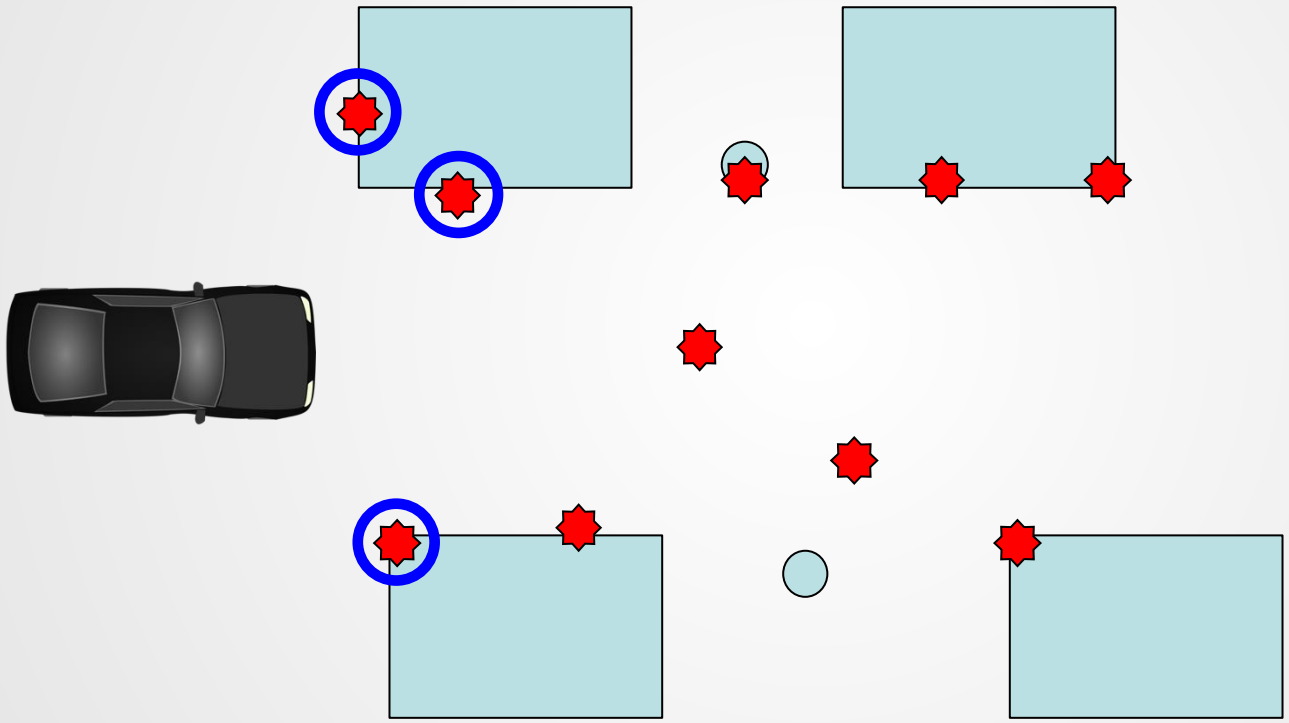



**The *local* map uses the platform to build itself**

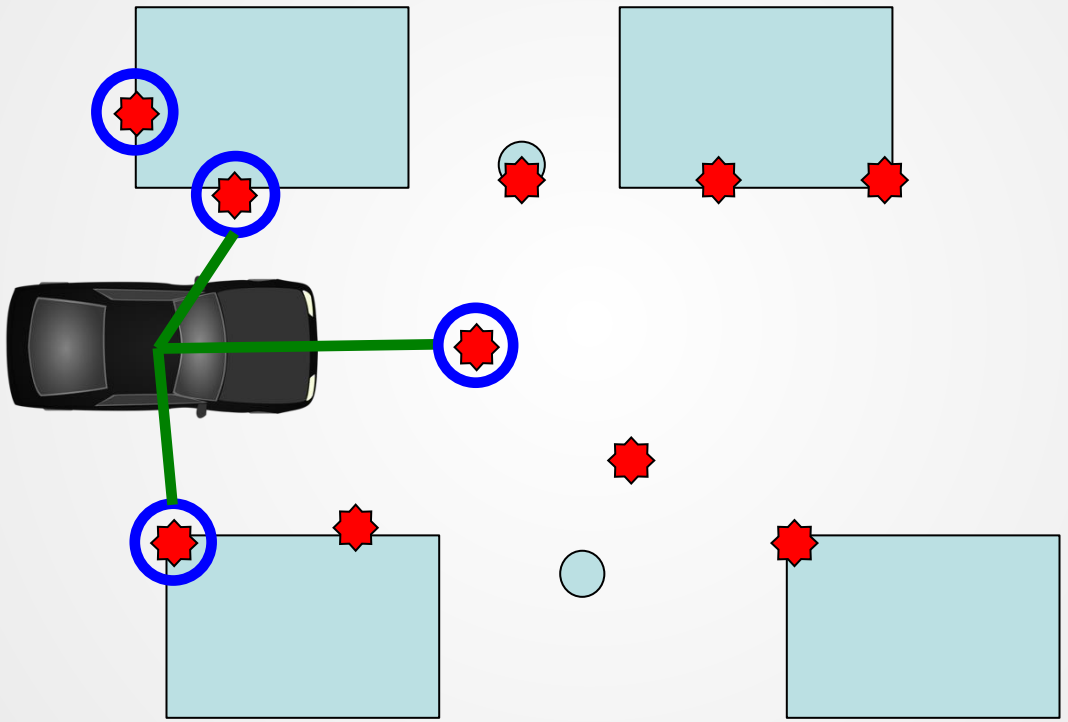
# Illustrating Odometry



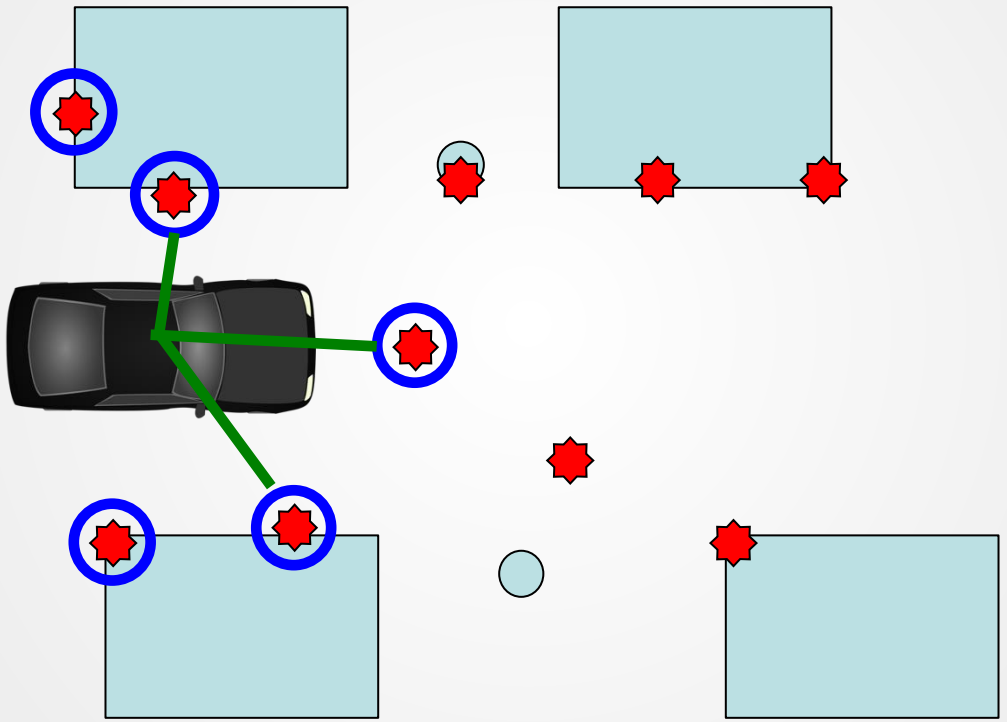
# Illustrating Odometry



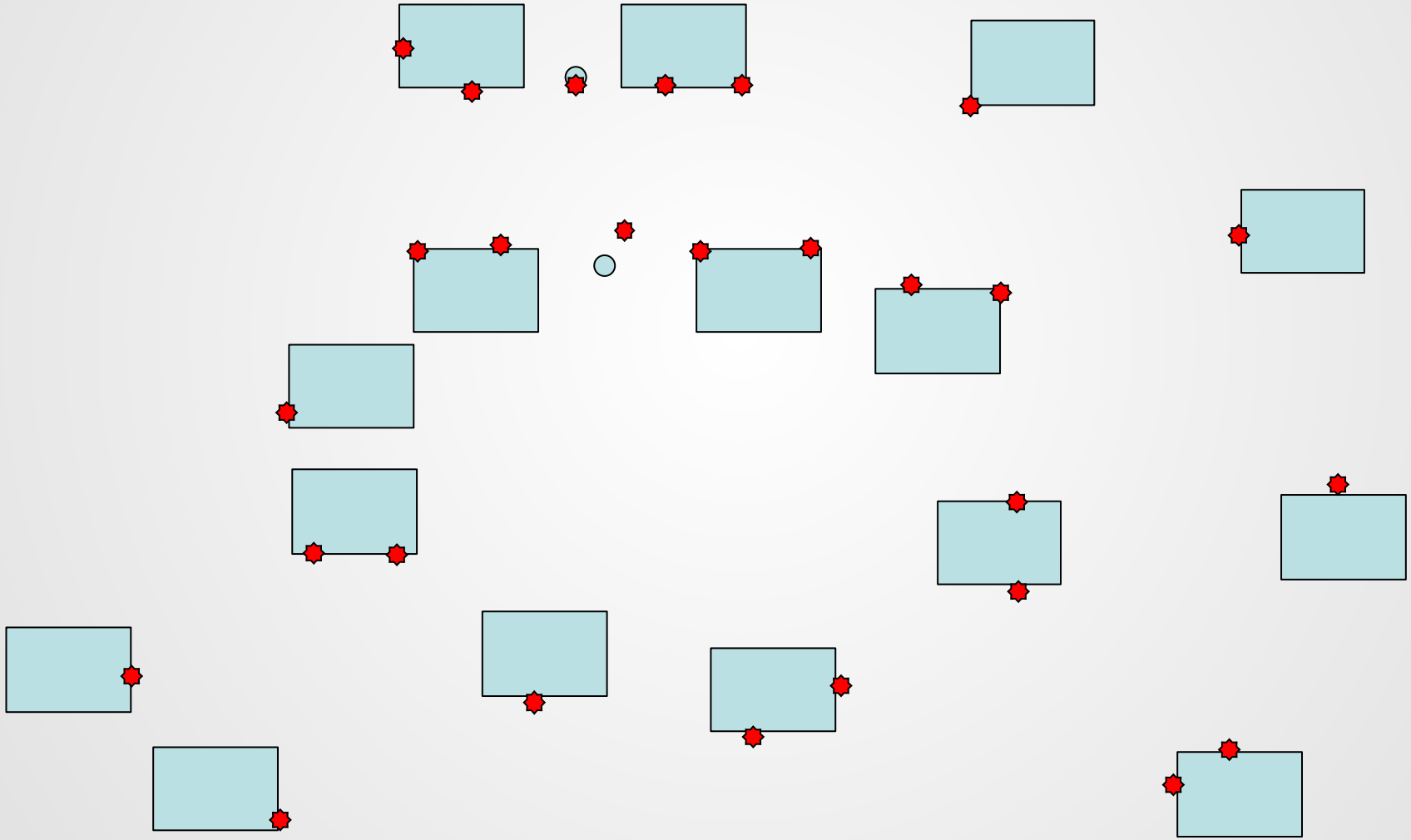
# Illustrating Odometry



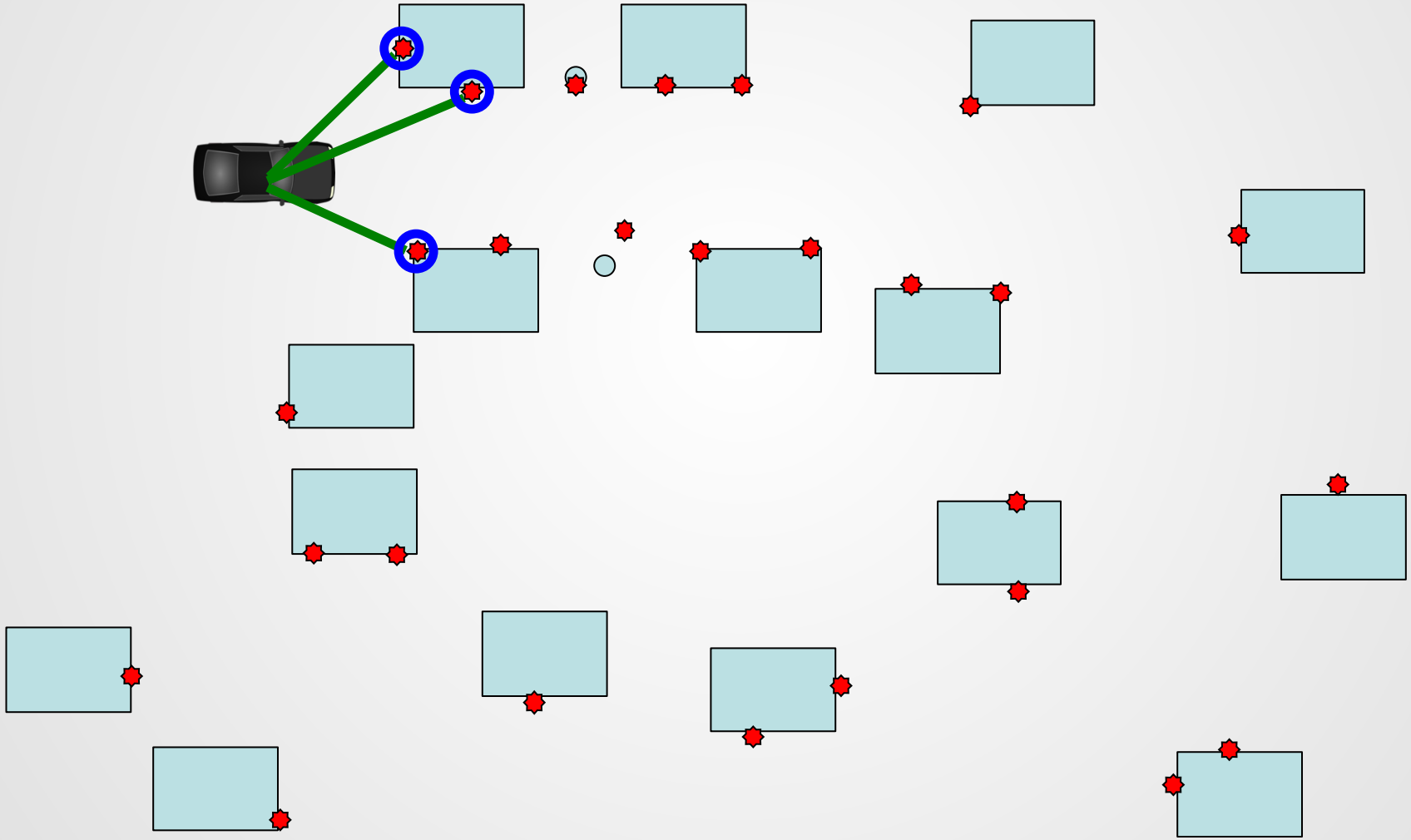
# Illustrating Odometry



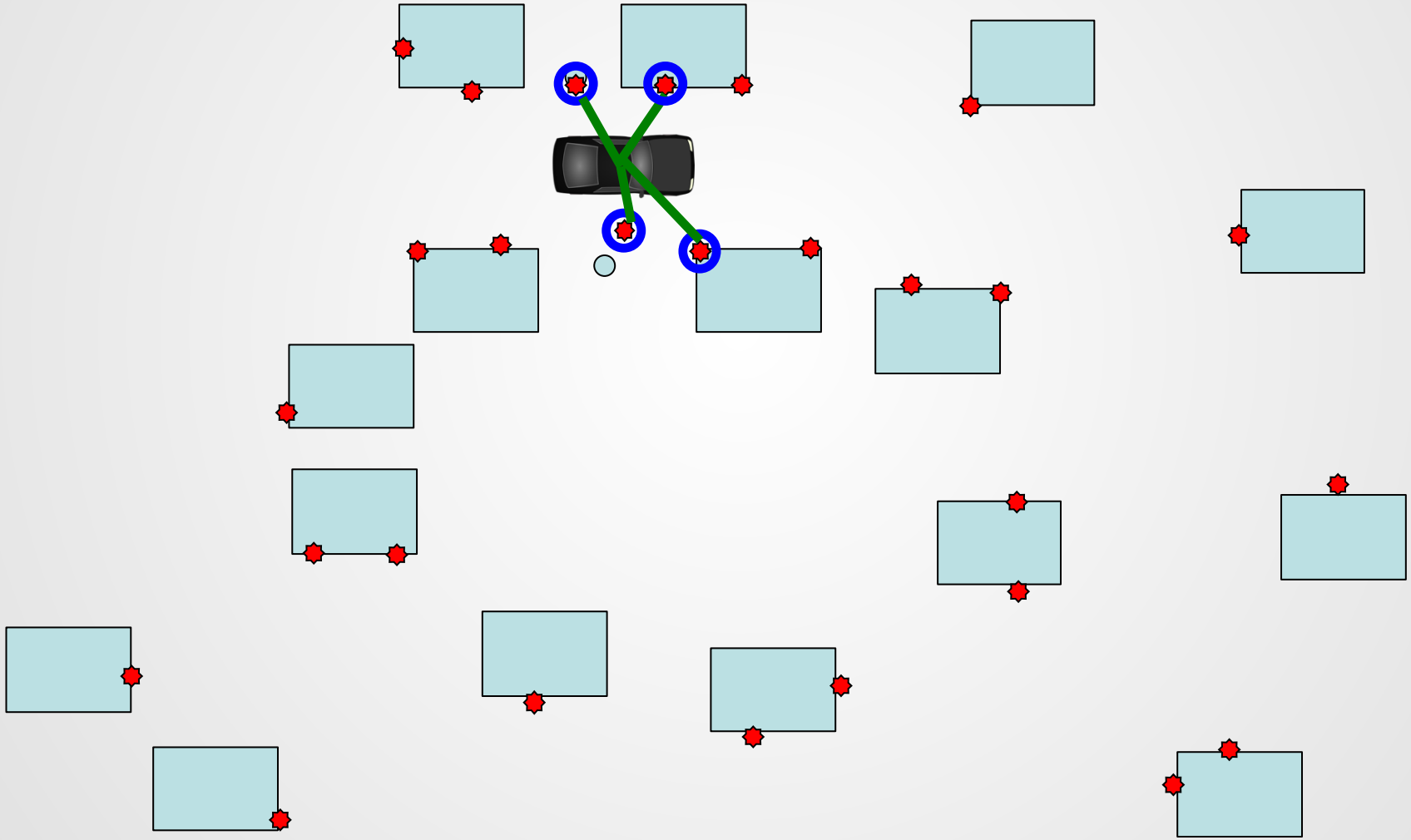
# High-Level View of Odometry



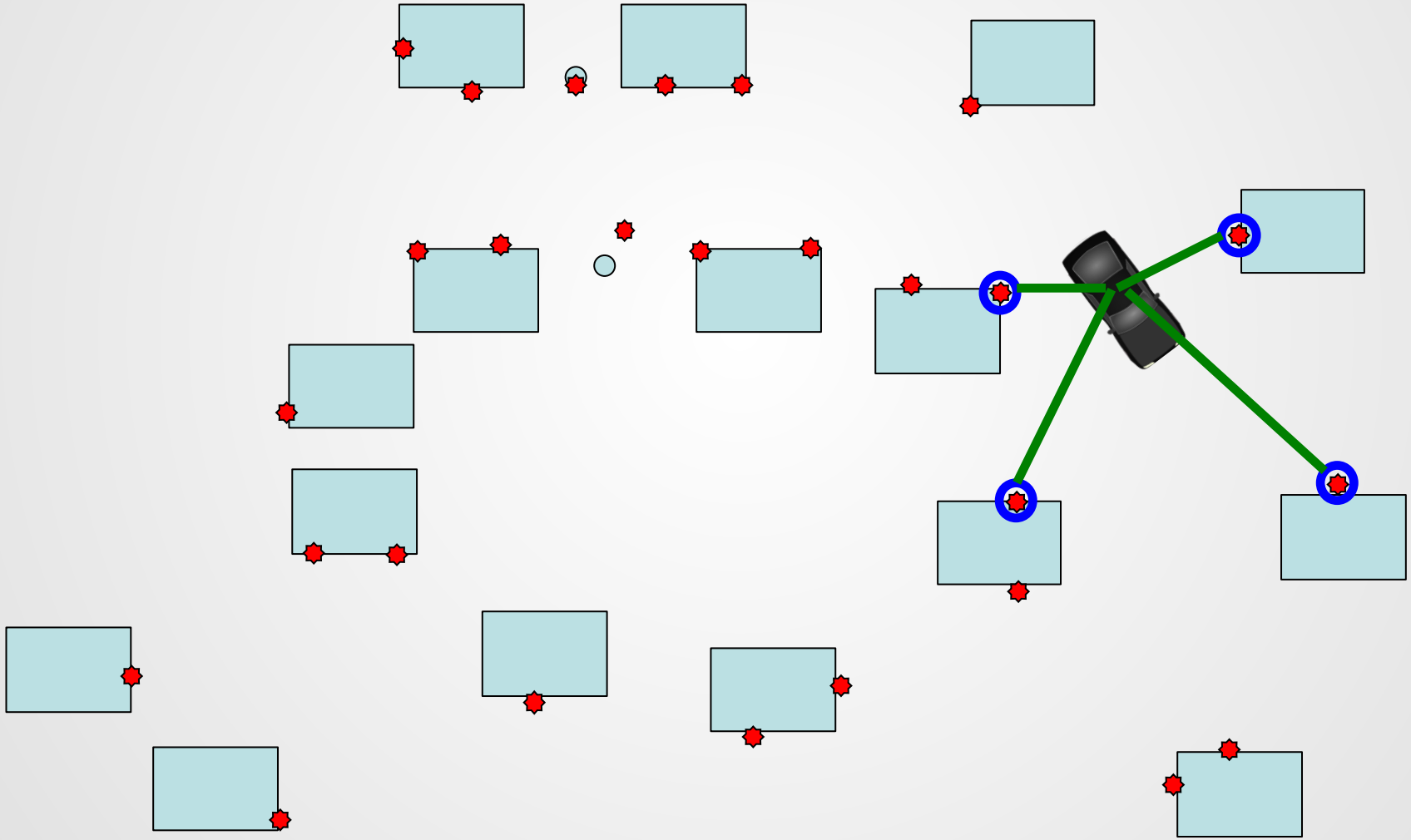
# High-Level View of Odometry



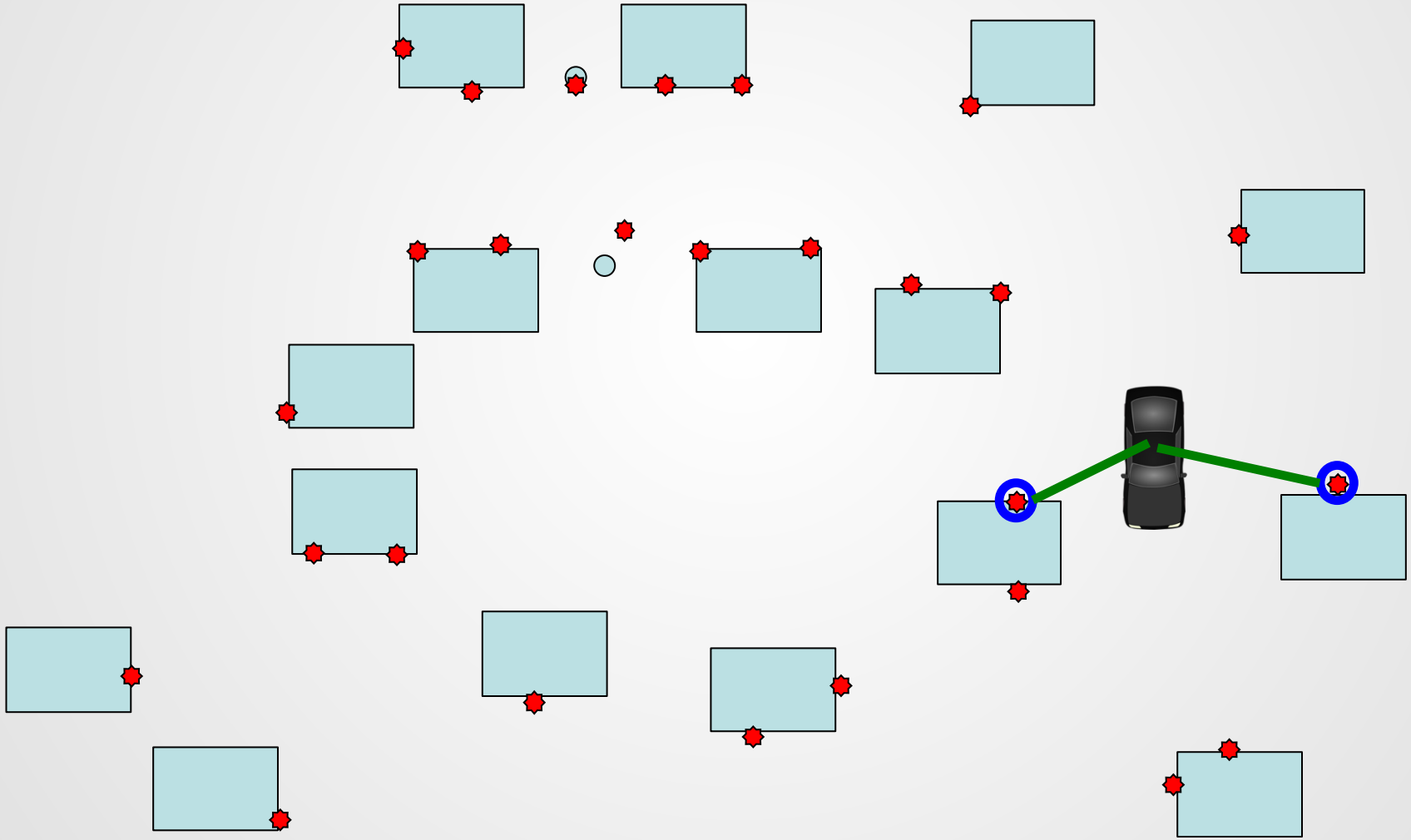
# High-Level View of Odometry



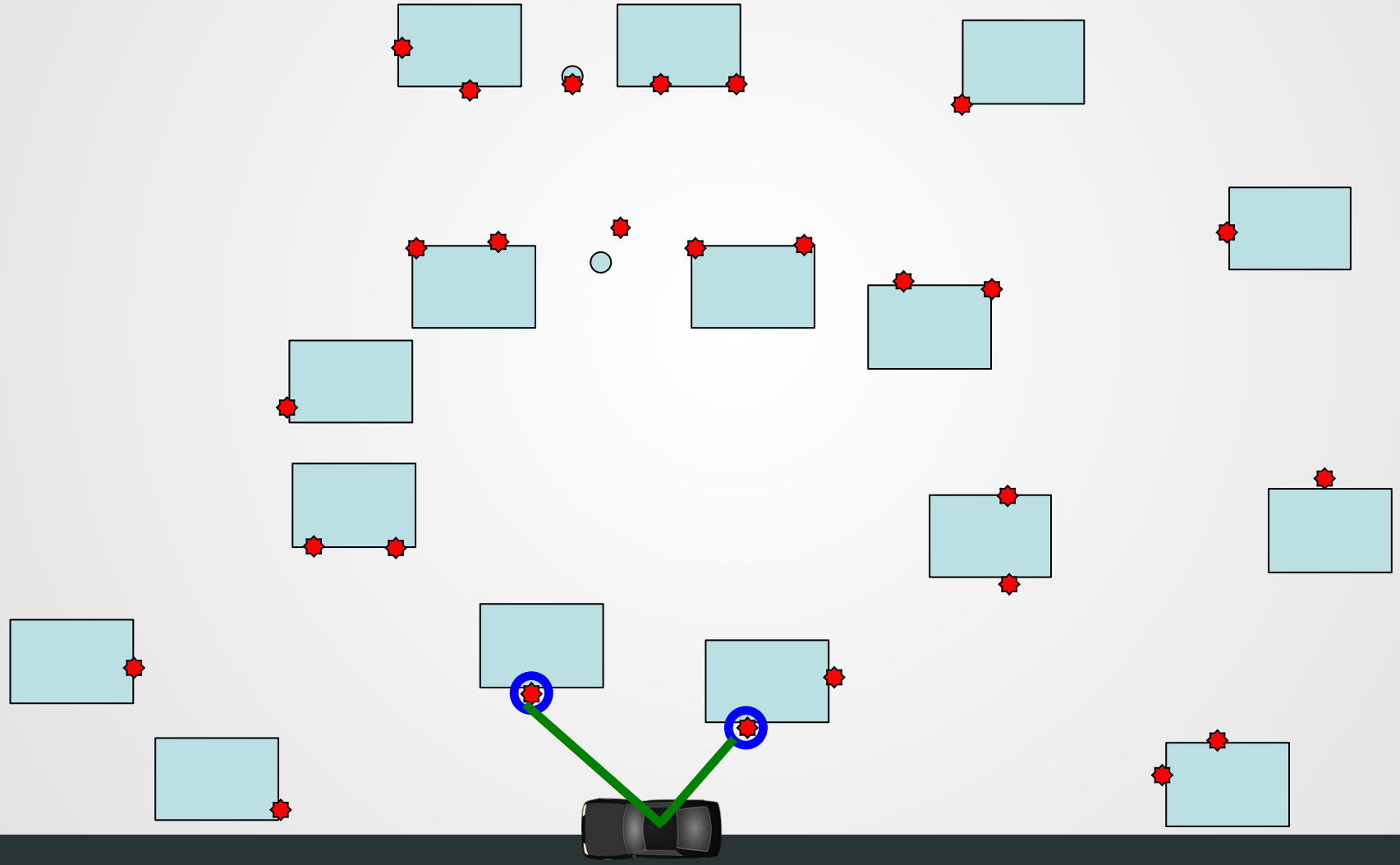
# High-Level View of Odometry



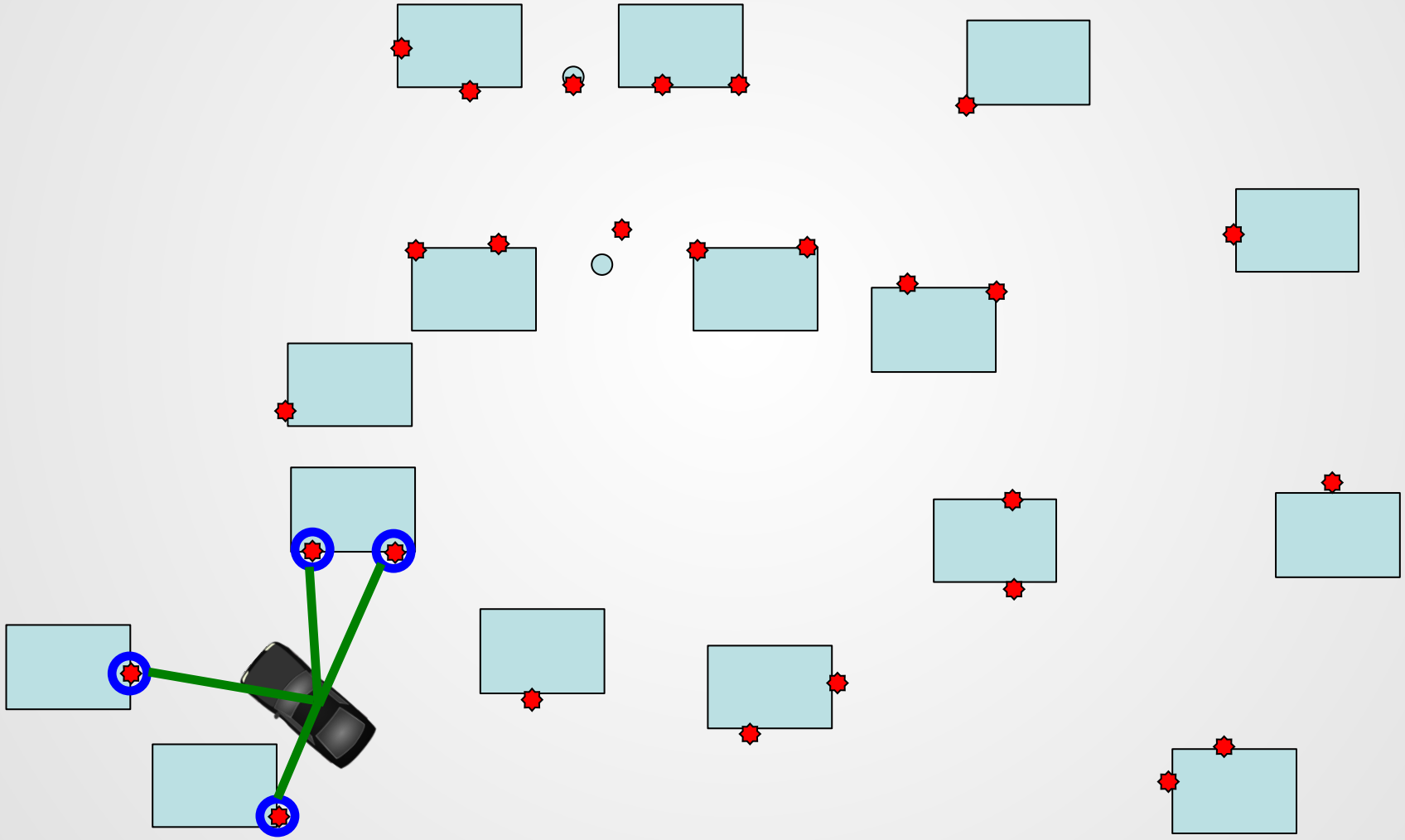
# High-Level View of Odometry



# High-Level View of Odometry

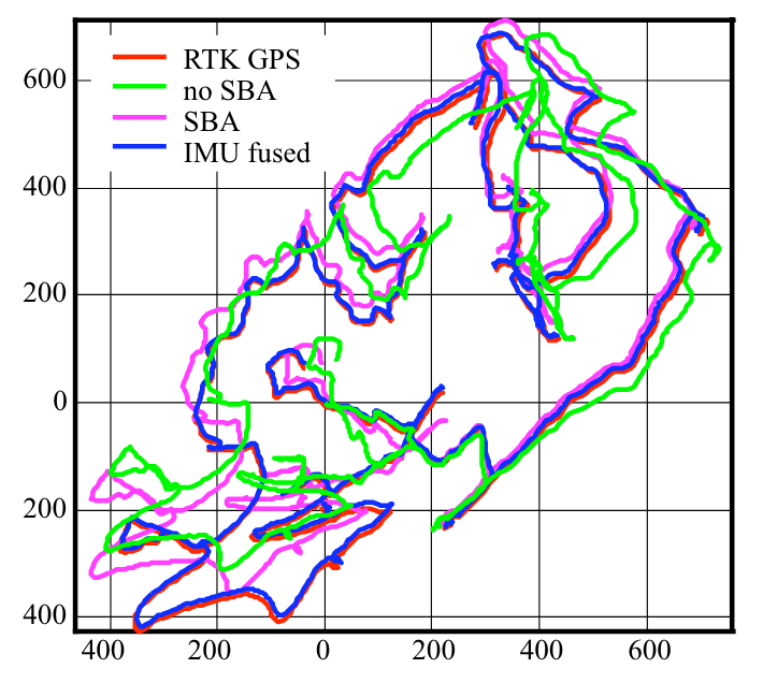


# High-Level View of Odometry



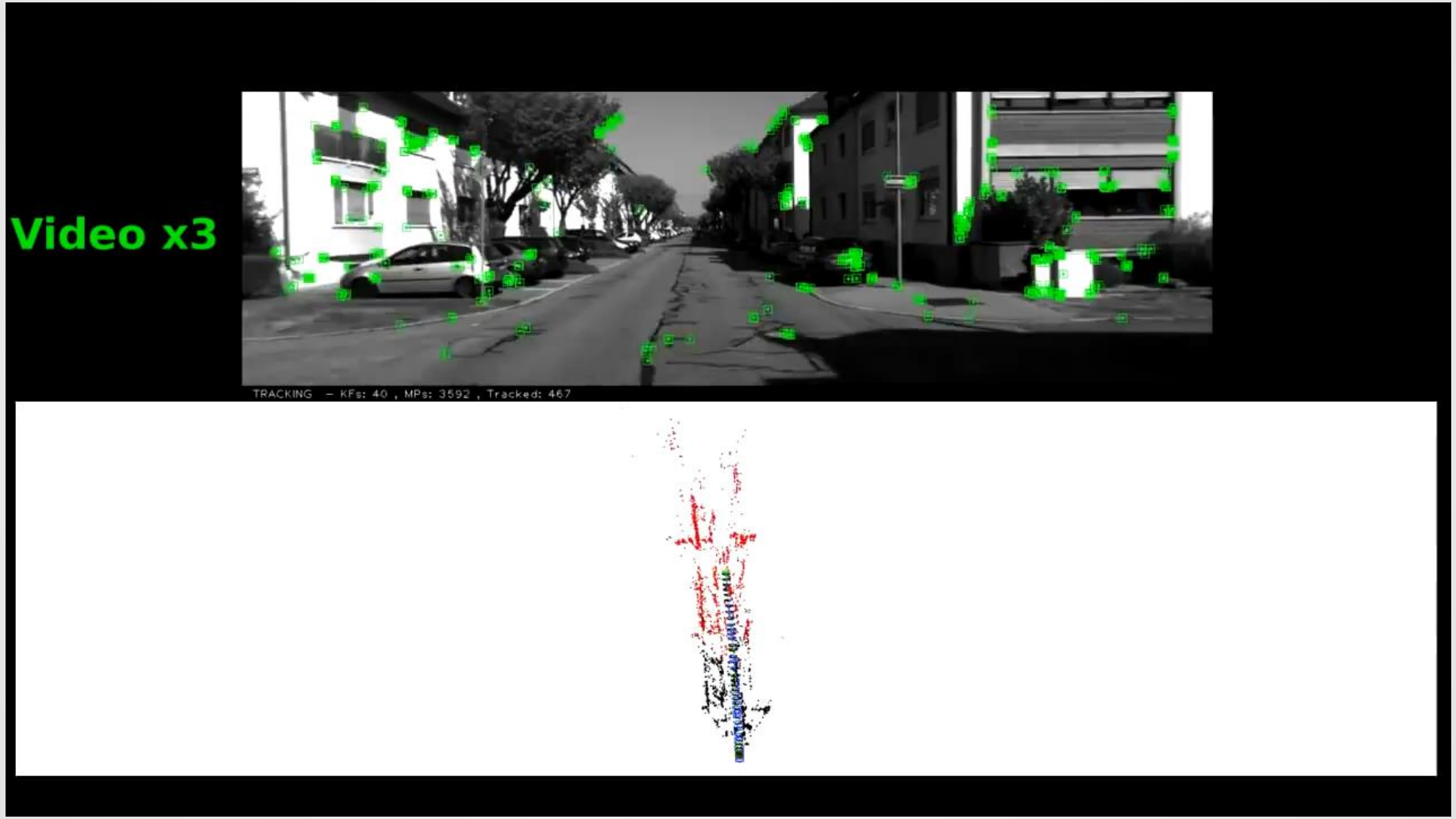
# Visual Odometry Can be Highly Effective

Rough Terrain Visual Odometry,
M. Agrawal, K. Konolige, 2007



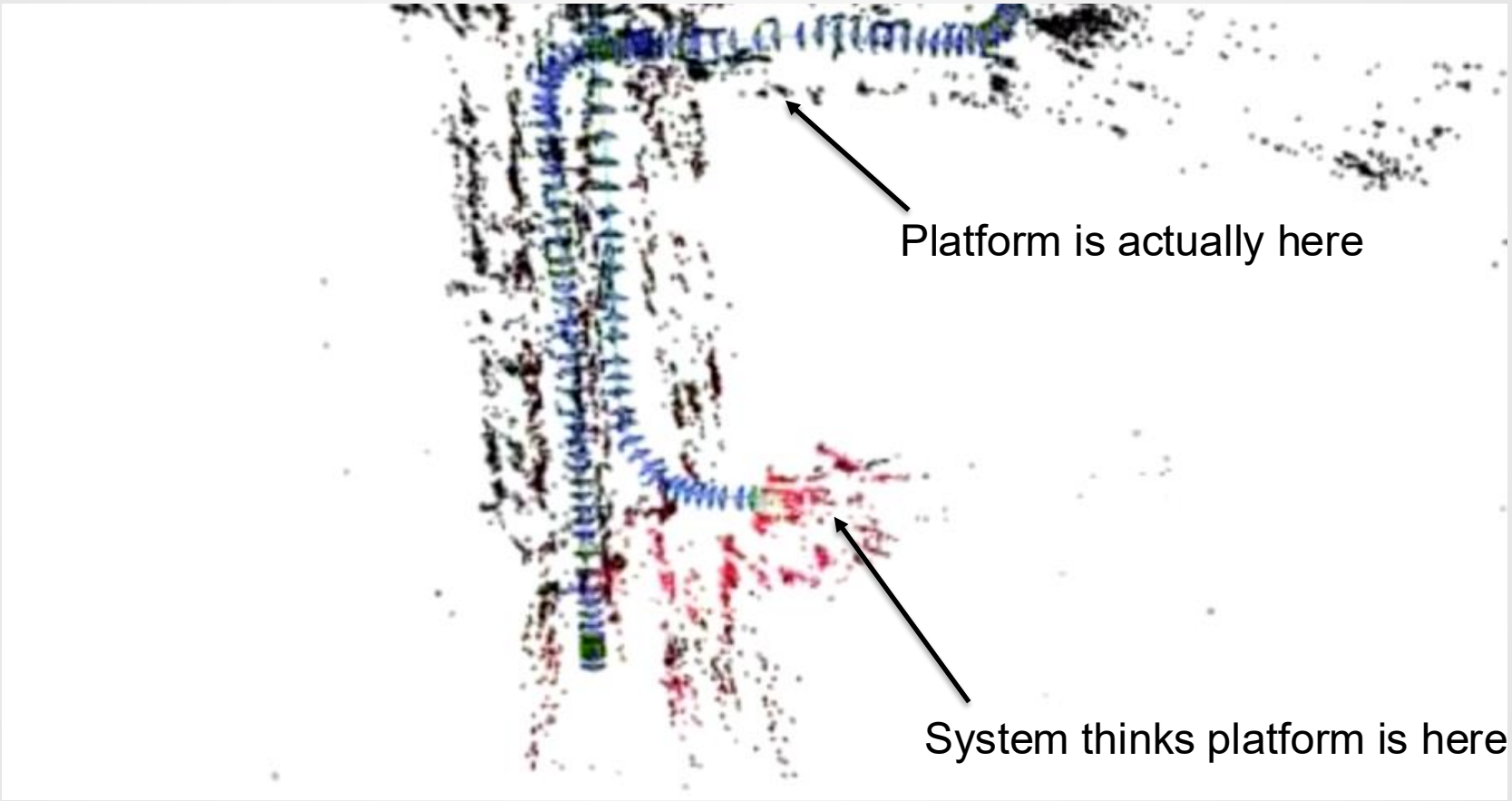

| | | RMS error in XYZ |
|---|---|---|
| Little Bit | VO No SBA | 97.41 (1.0%) |
| | VO SBA | 45.74 (0.49%) |
| | VO No SBA + IMU | 7.83 (0.08%) |
| | VO SBA + IMU | 4.09 (0.04%) |

# ORB-SLAM Running as Visual Odometry



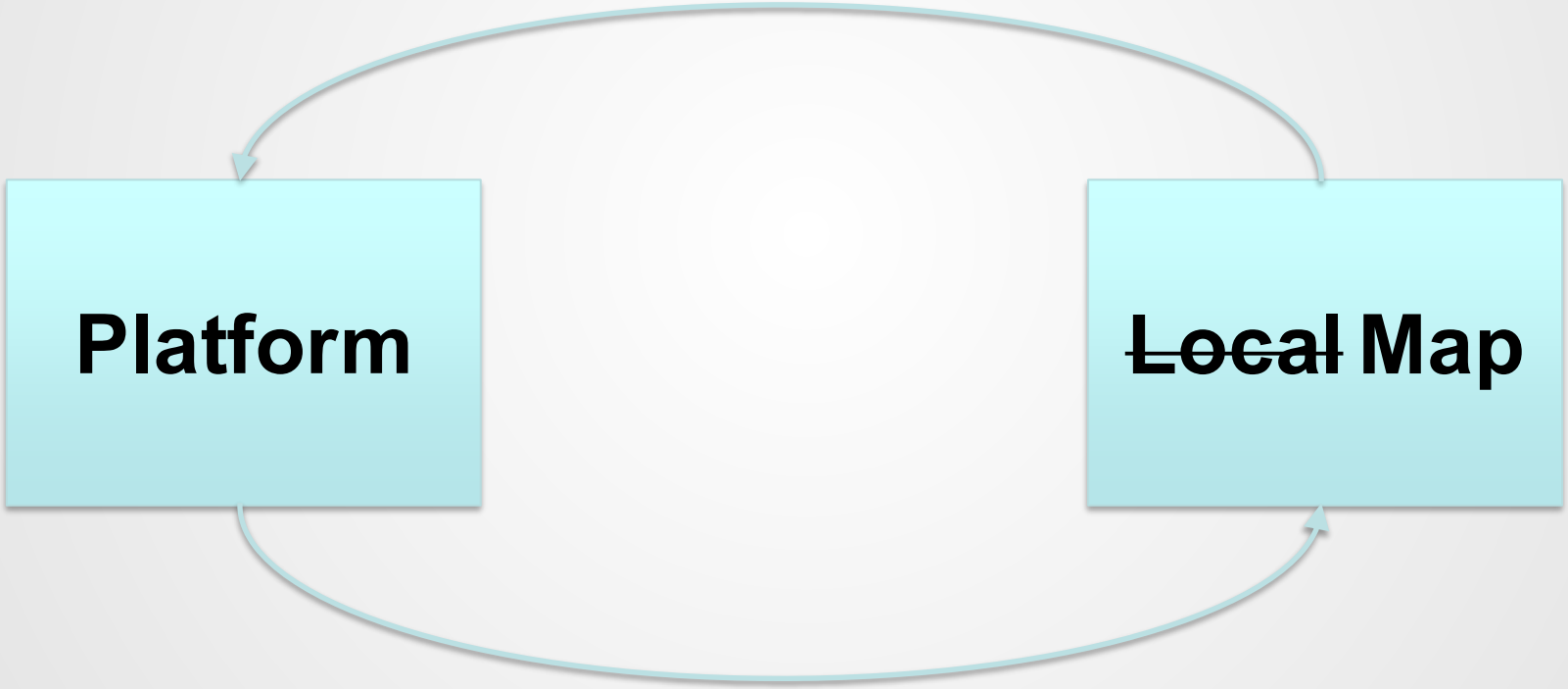

From https://www.youtube.com/watch?v=8DISRmsO2YQ (00:15-00:59.4 seconds)

# Drift in ORB-SLAM



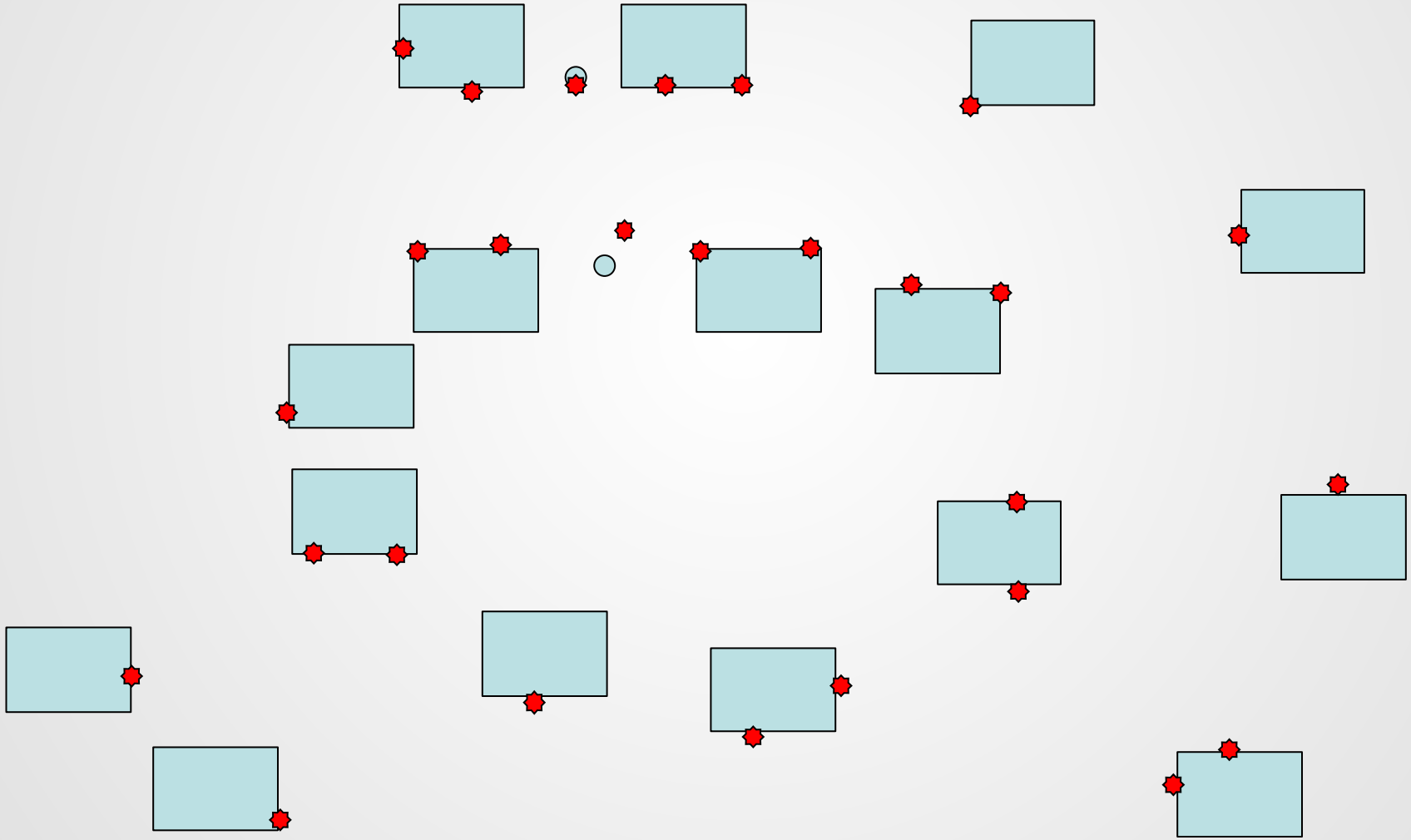
# SLAM

The platform uses the ~~*local*~~ map to localise

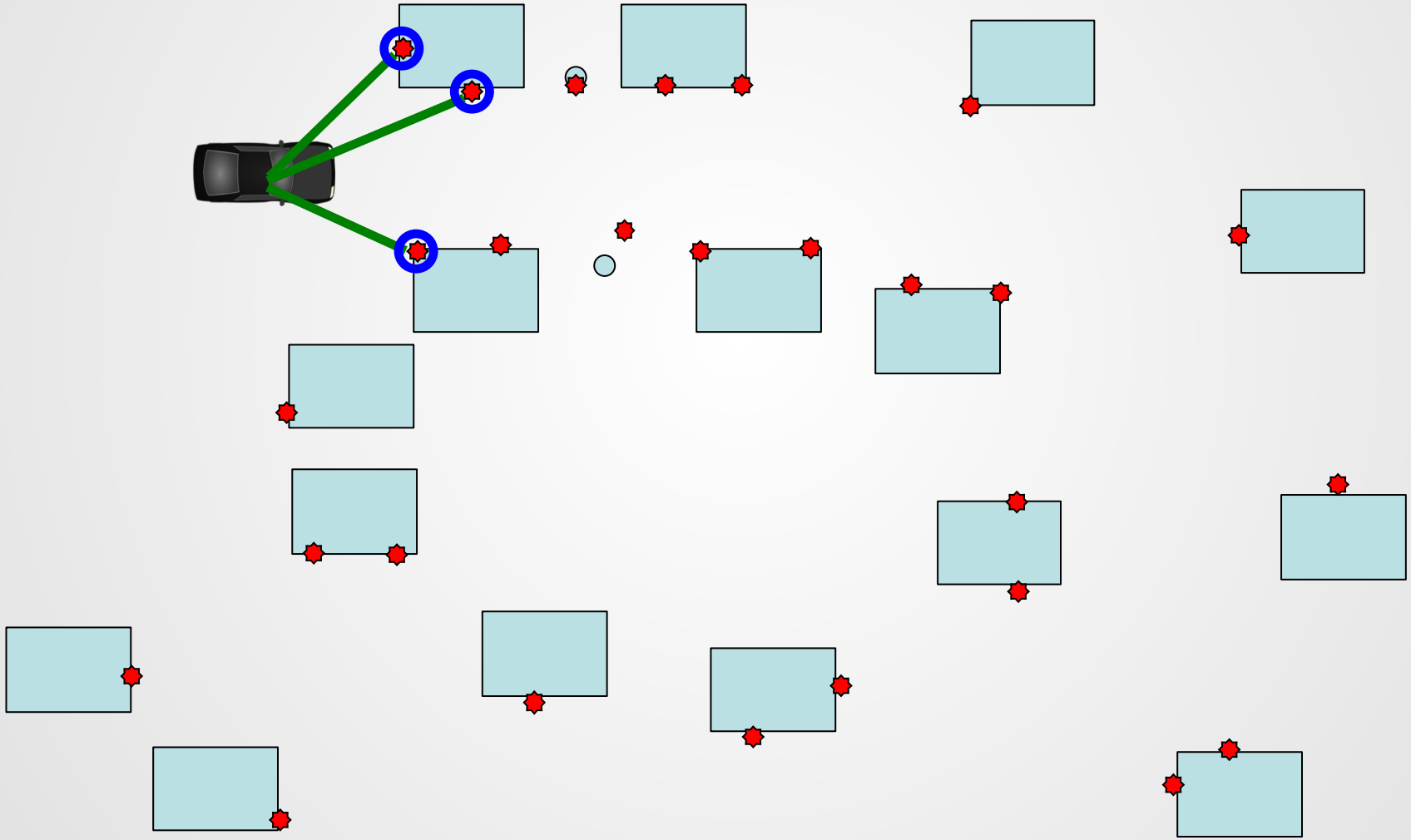



The ~~*local*~~ map uses the platform to build itself

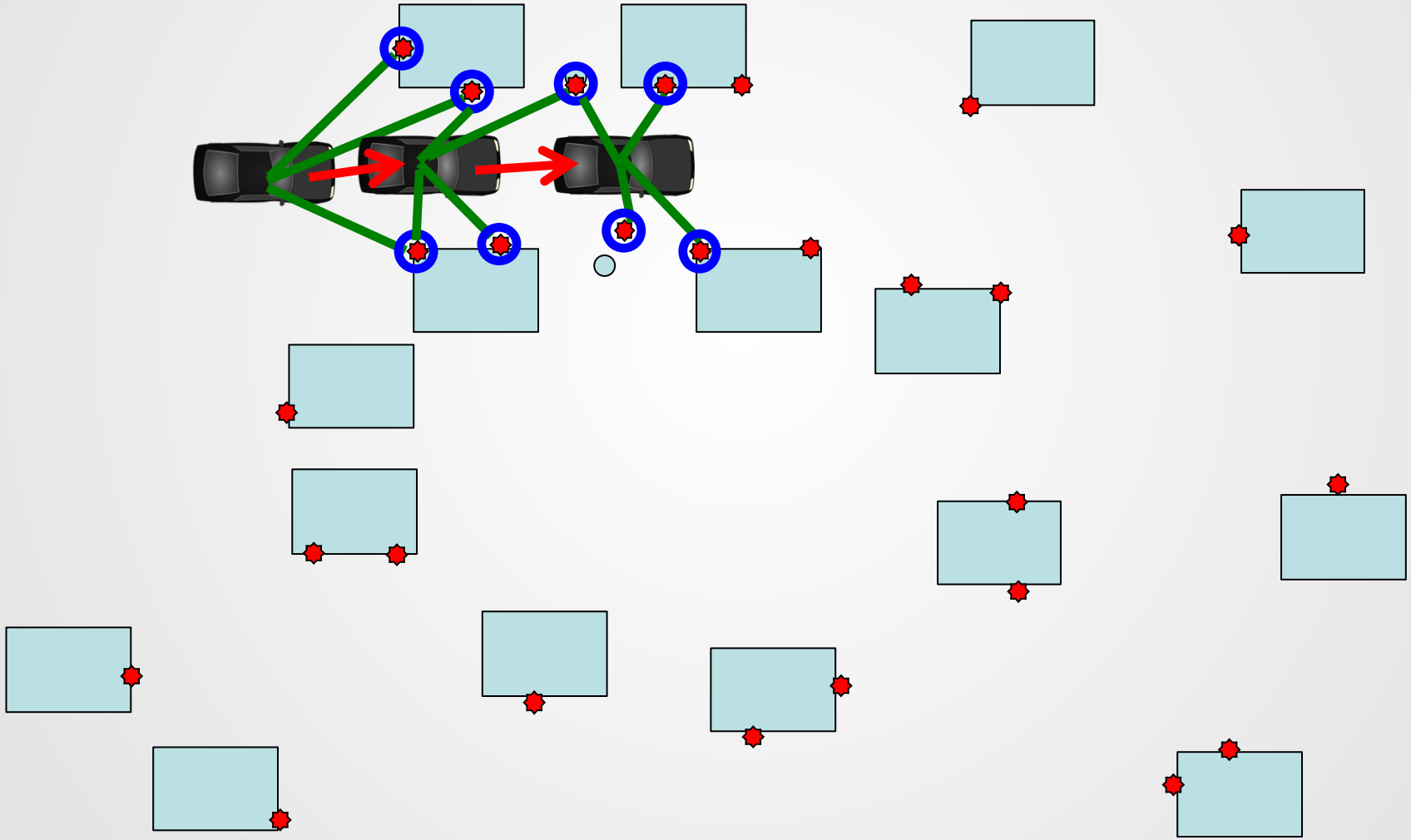
# High-Level View of Self-Mapping (SLAM)



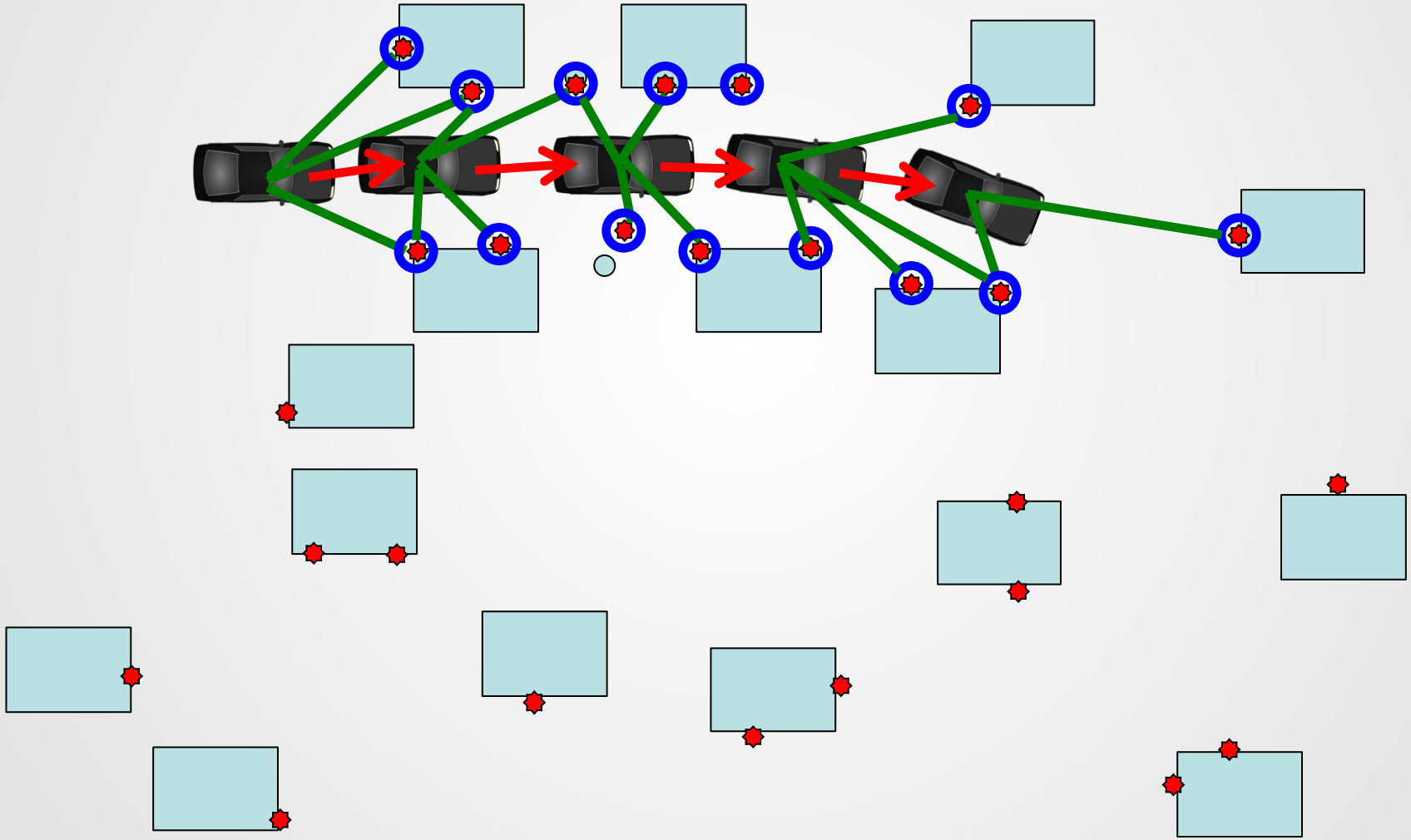
# High-Level View of Self-Mapping (SLAM)



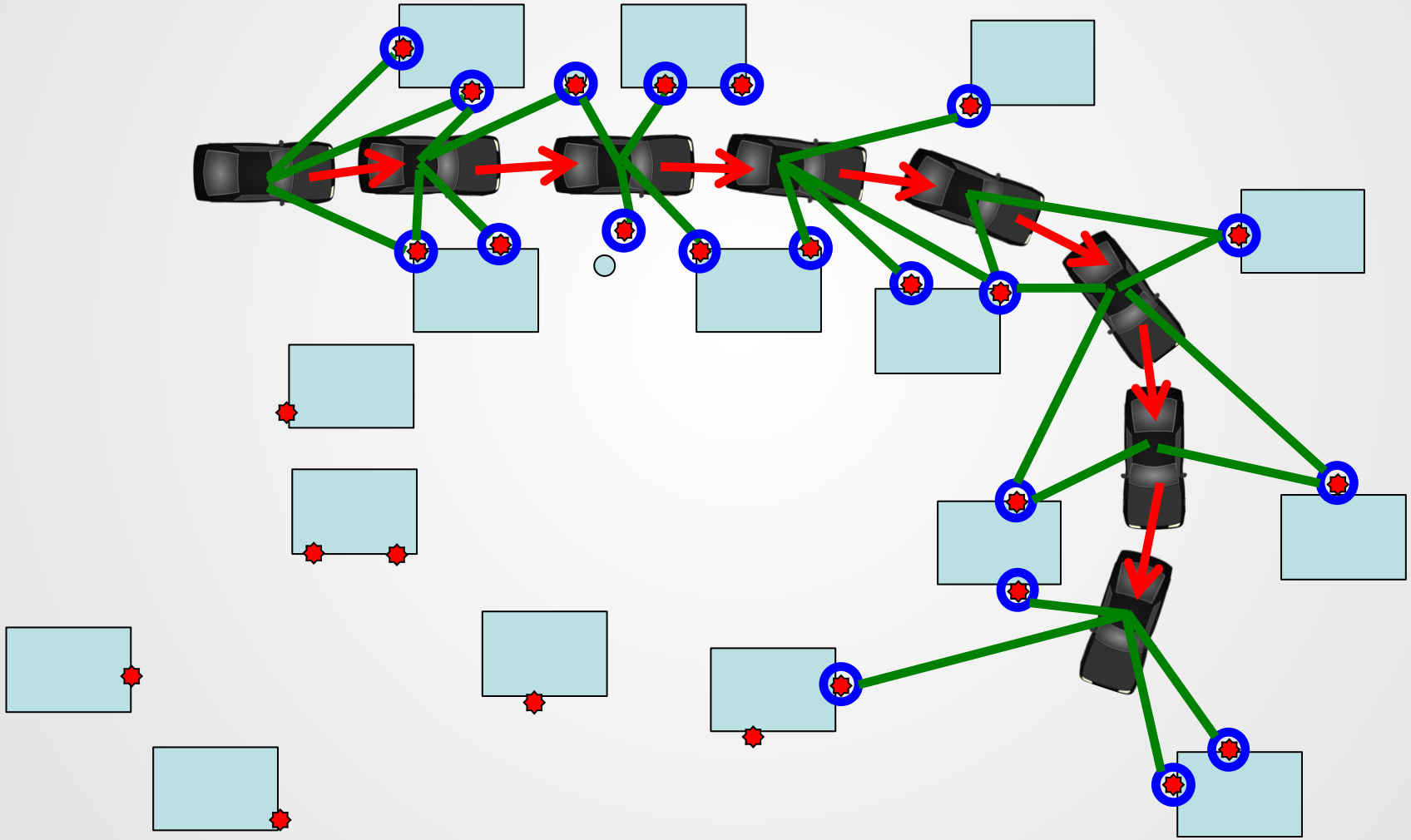
# High-Level View of Self-Mapping (SLAM)



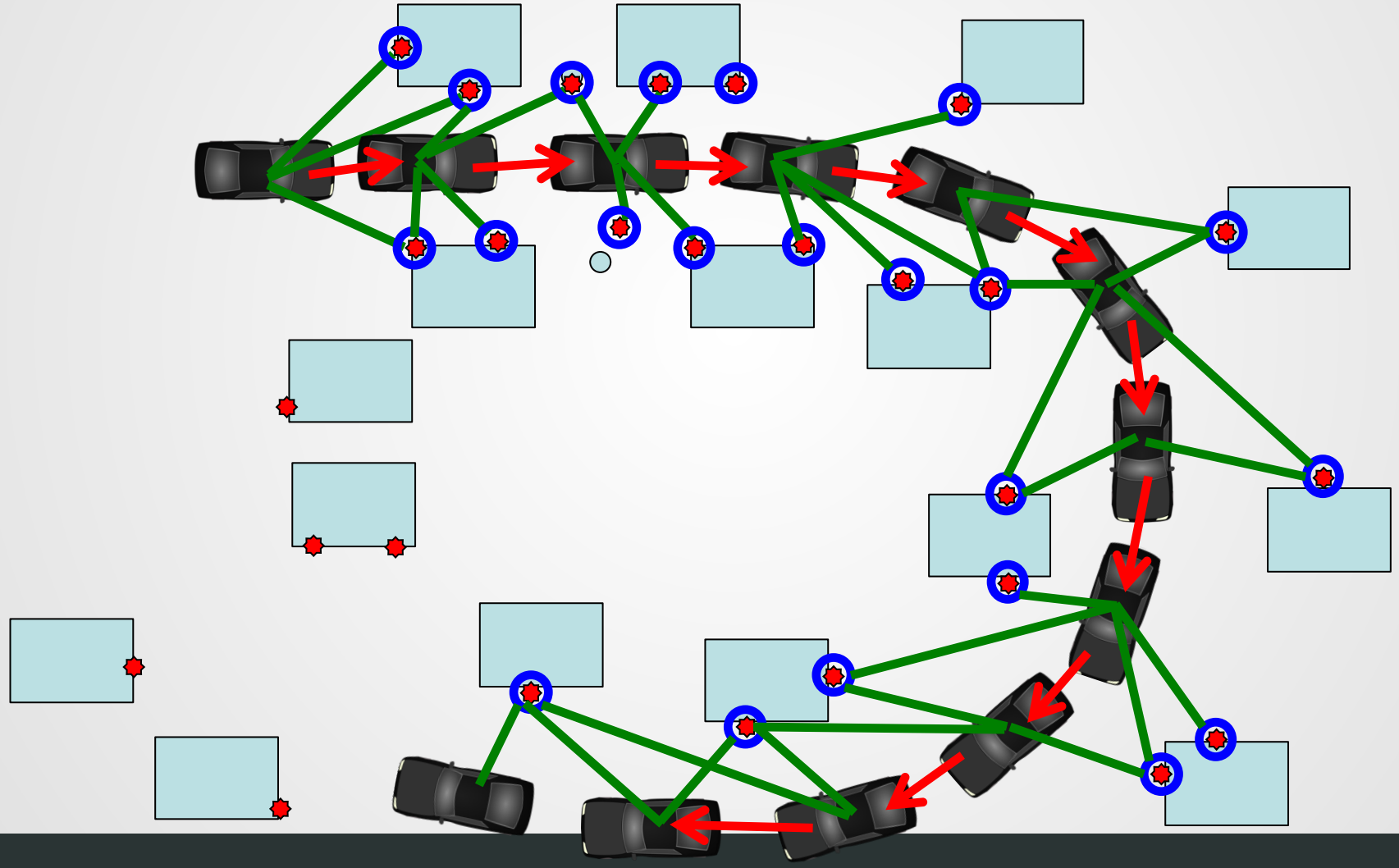
# High-Level View of Self-Mapping (SLAM)



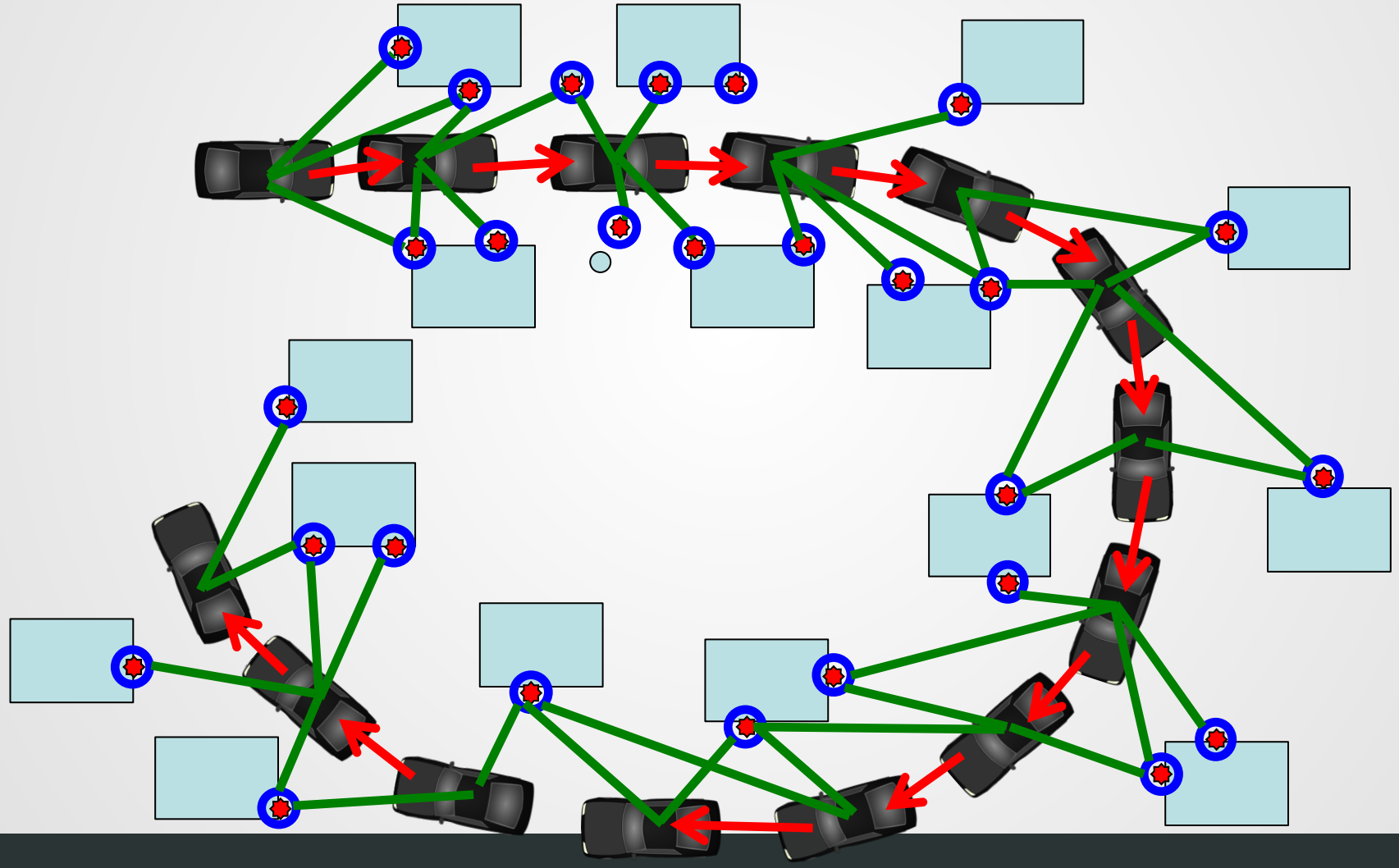
# High-Level View of Self-Mapping (SLAM)



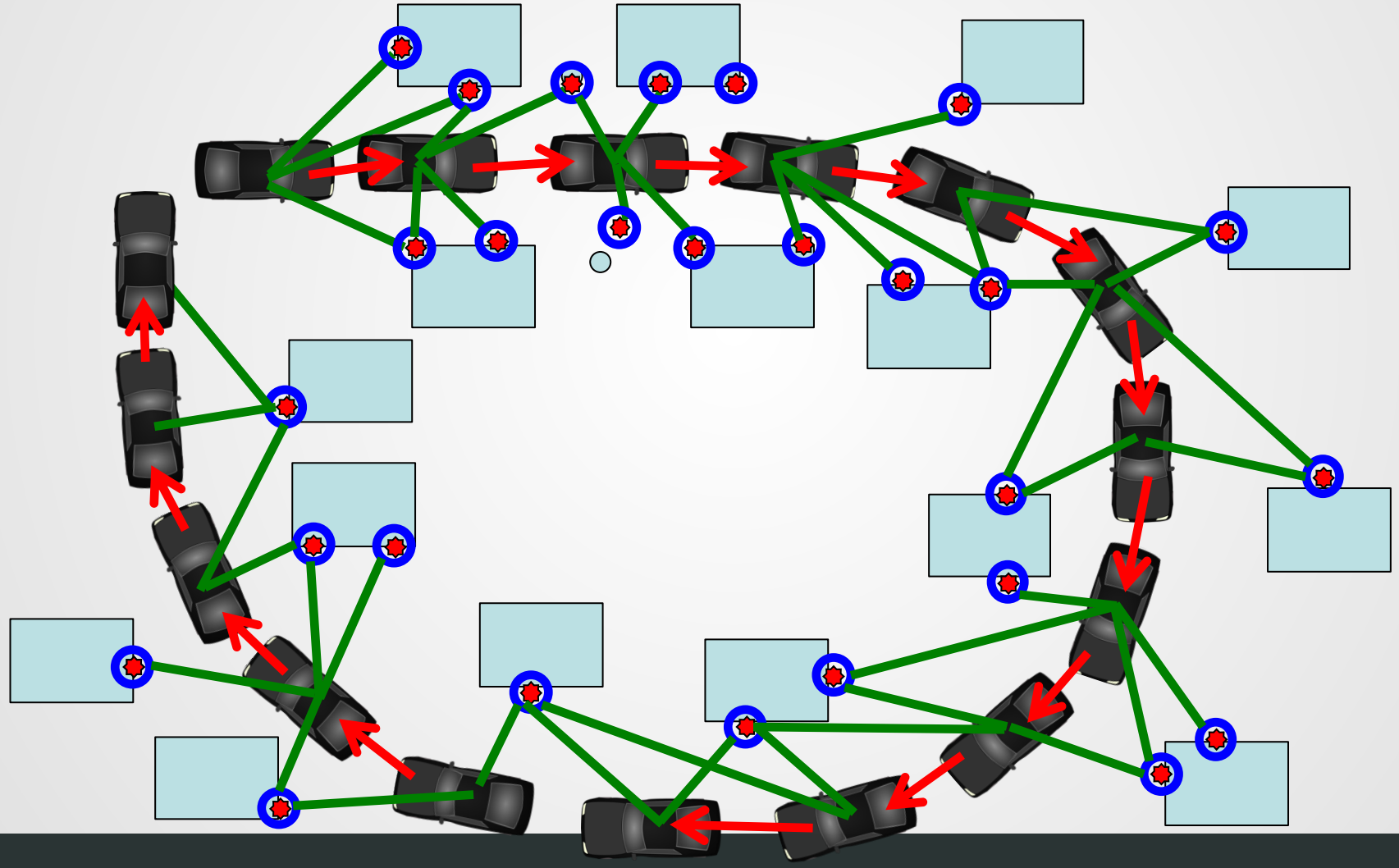
# High-Level View of Self-Mapping (SLAM)



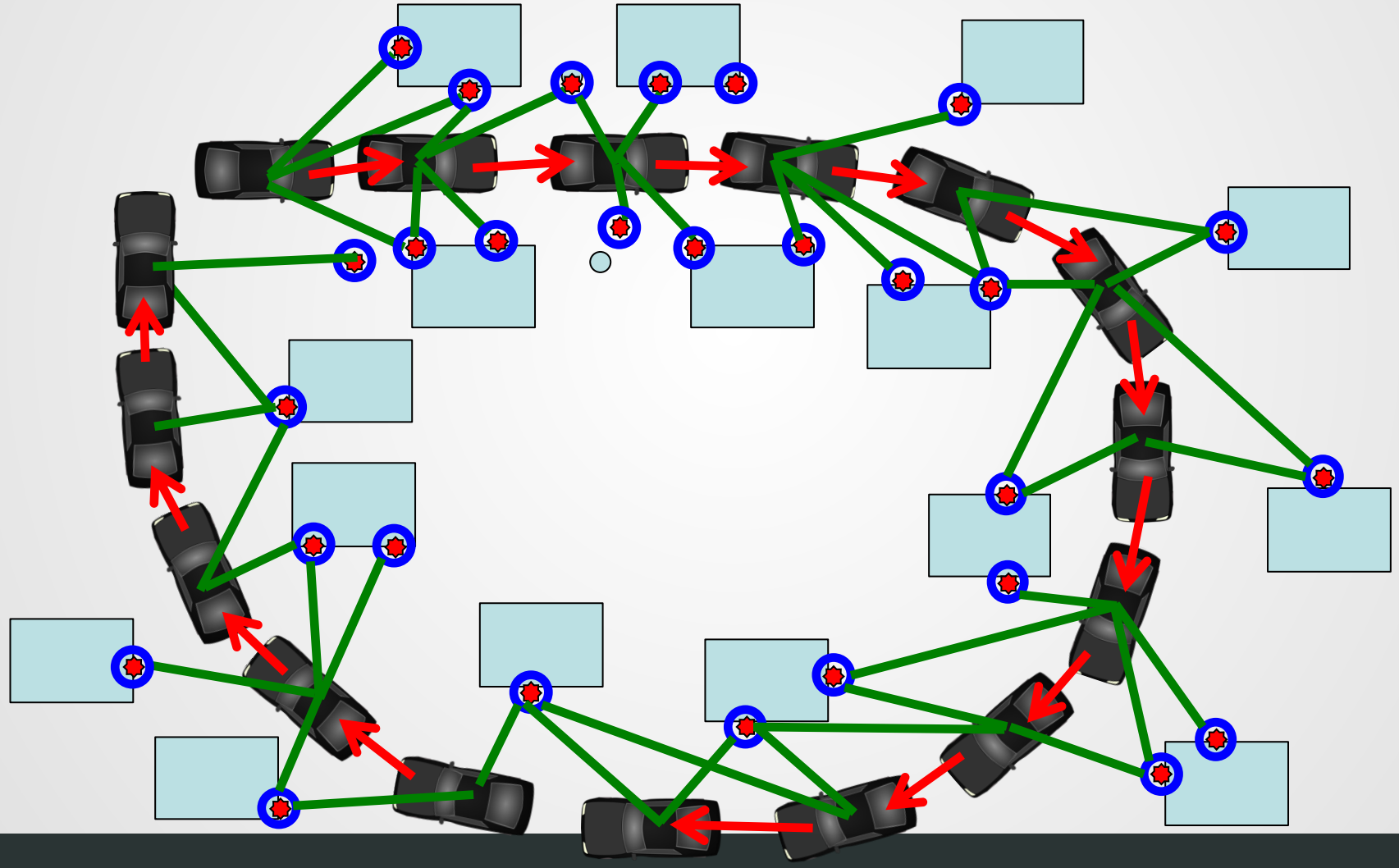
# High-Level View of Self-Mapping (SLAM)



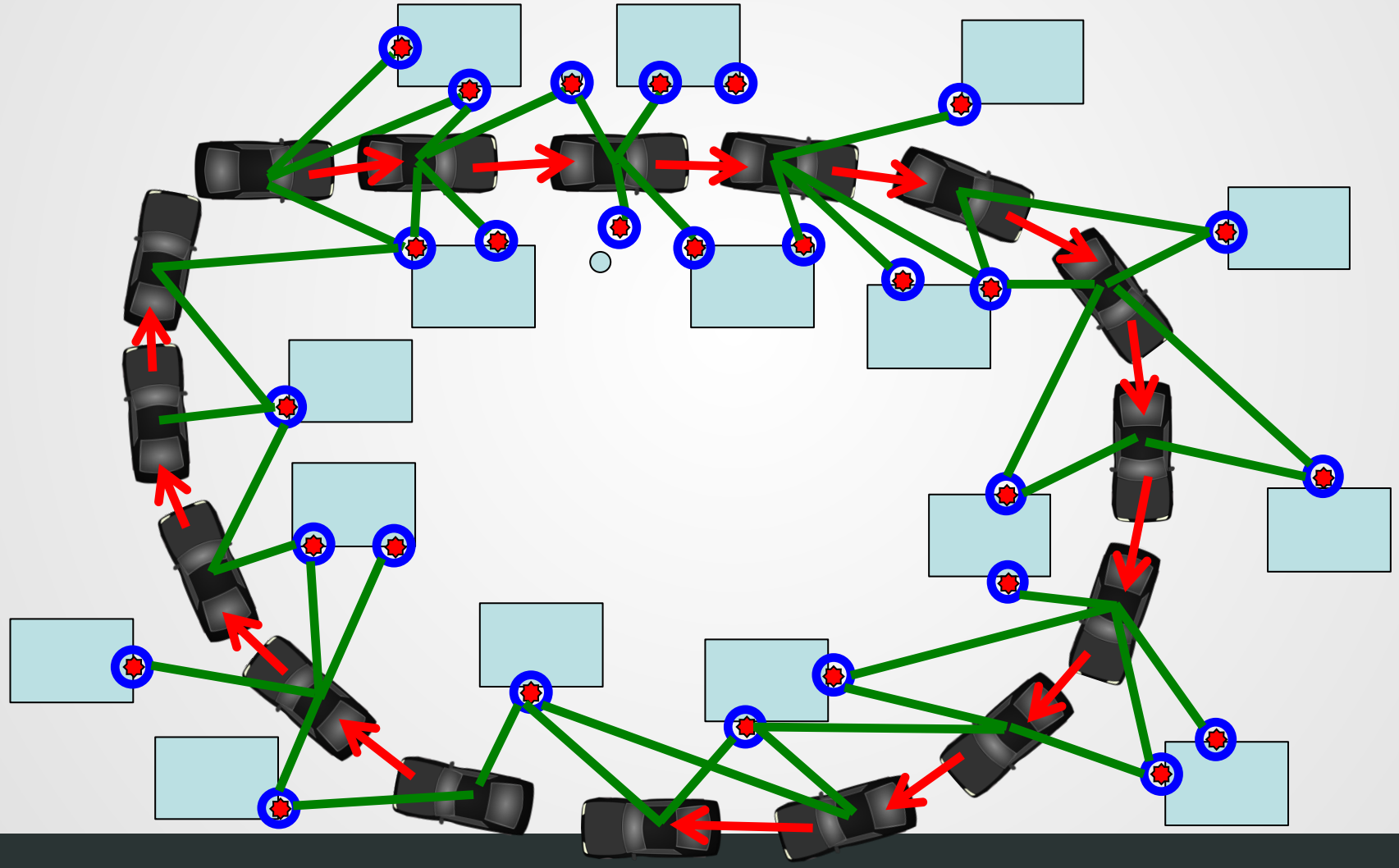
# High-Level View of Self-Mapping (SLAM)



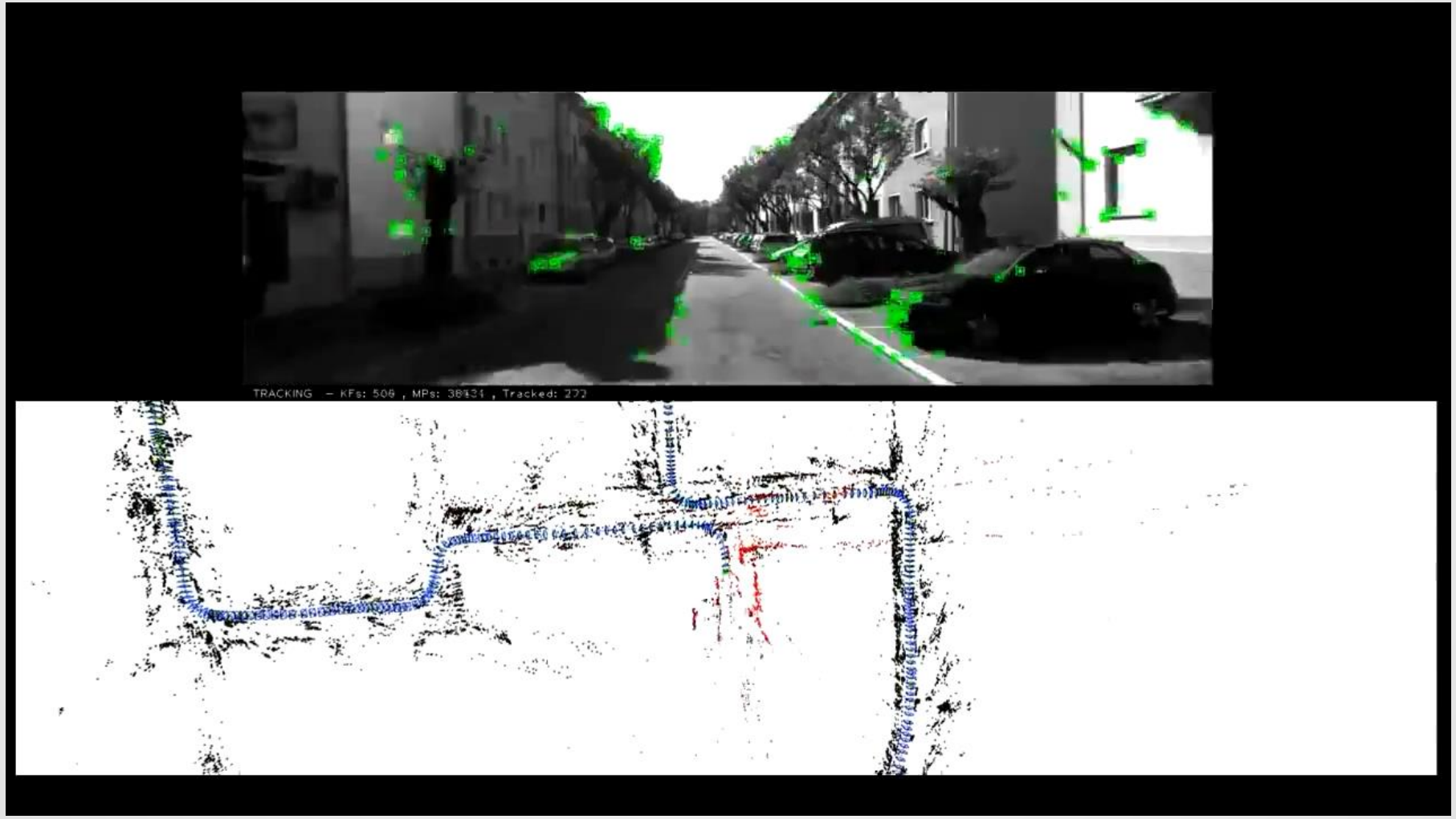
# High-Level View of Self-Mapping (SLAM)



# SLAM and Loop Closing



# Loop Closure in ORB-SLAM on KITTI




From https://www.youtube.com/watch?v=8DISRmsO2YQ (00:57.5-00:62.5 seconds)

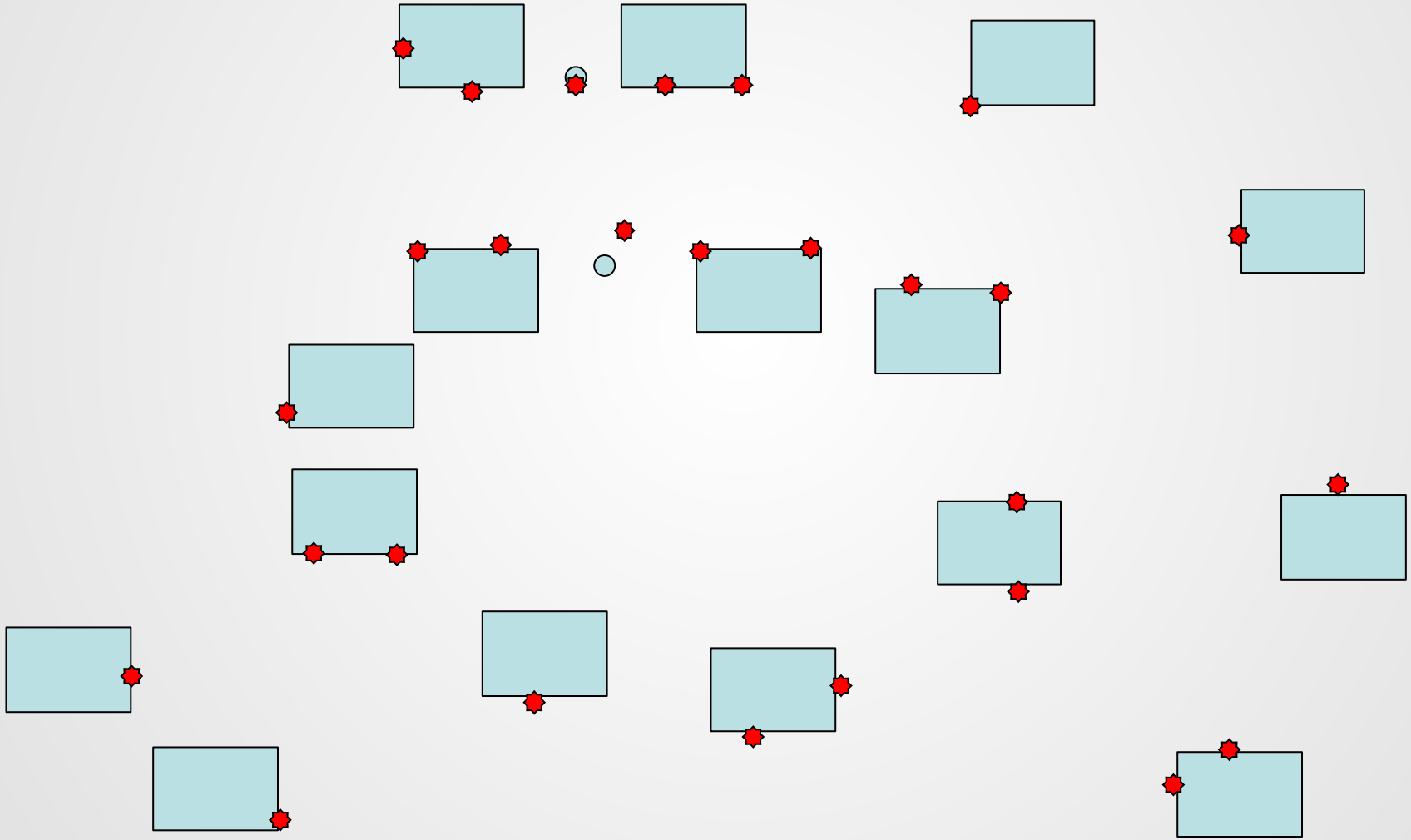
# Sparse Feature Point Maps

- *Localization, Mapping, Odometry, SLAM*
- Sparse Feature Point Maps
- *Notation*

# Individual Map Points

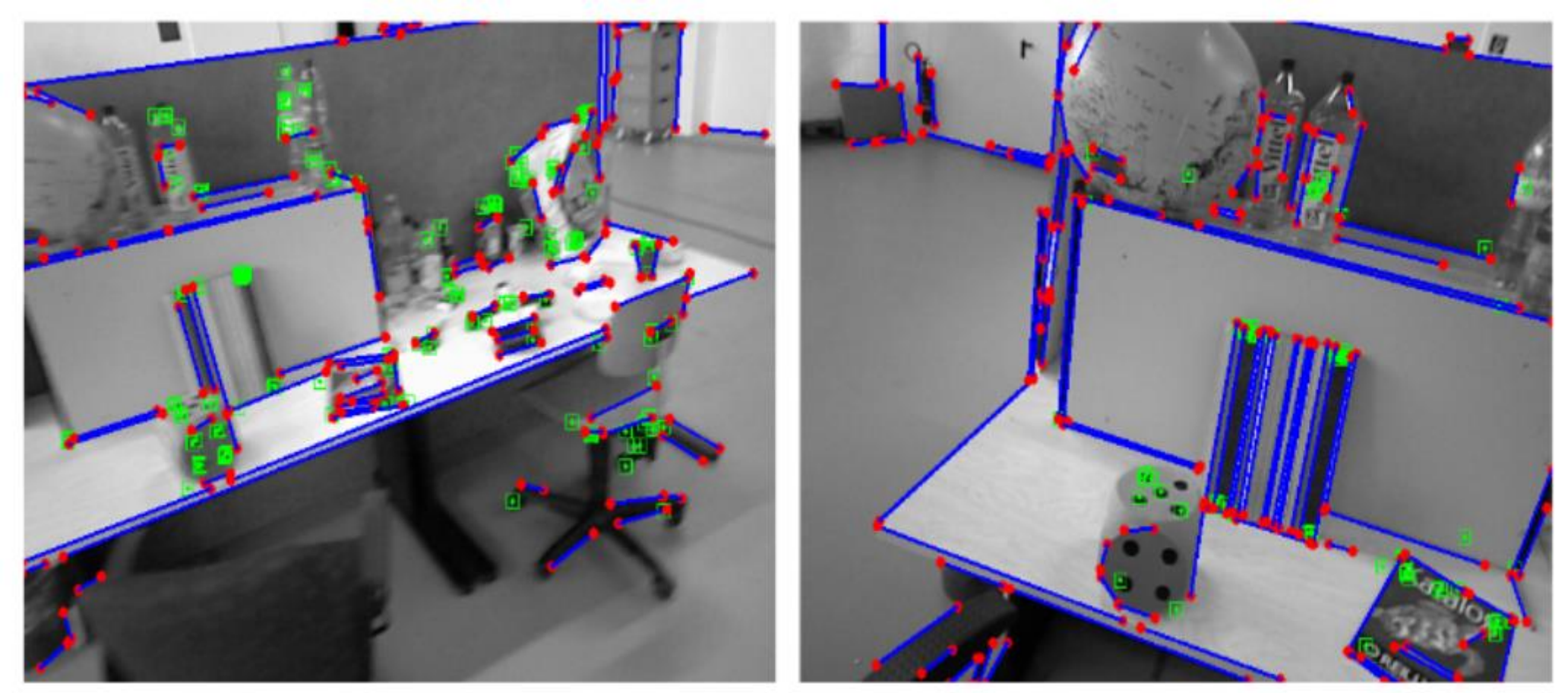
- The idea is that we represent the environment by a sparse set of features
- The features are assumed to be distributed randomly
- Therefore, there are no semantics or other “high-level” information

# Environment with Landmarks



frame 1
key 0
keyframes 1
from start 0.001m
covered 0.000m
inliers 319
outliers 10
time per frame 34ms
FeatureDetectorFast
DescriptorSchemeSAD

# Points and Edges



PL-SLAM: Real-Time Monocular Visual SLAM with Points and Lines, Pumarola et al, 2017

# Characteristics of Landmarks

- ***They*** **must be** ***detectable*** **by the sensing systems and feature detection algorithms**
- **They must be** ***distinct*** **from one another so they can be individually distinguished**
- ***When observing them, spatial information can be obtained***

# Detectable

- The easiest case is when the targets are artificial
- They have some distinctive properties about them which makes them easy to detect
- They can provide additional information (e.g., numerical ID) through a coded target or through relative placement

# AprilTag

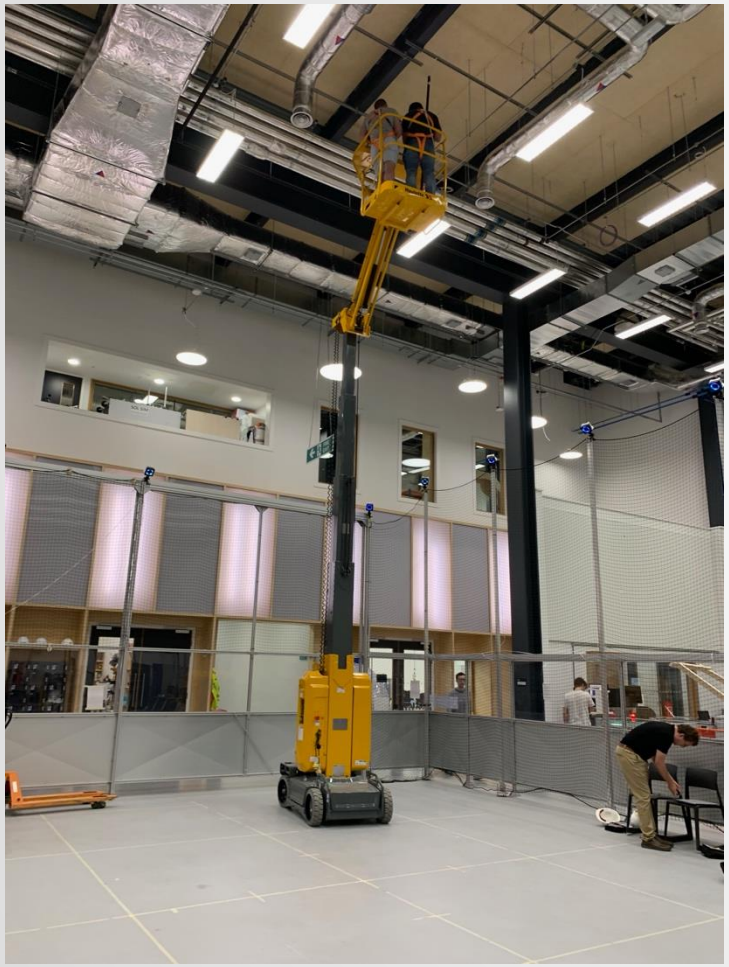
From "What is AprilTag?"

# AprilTag for Prototype VIVE Tracking System

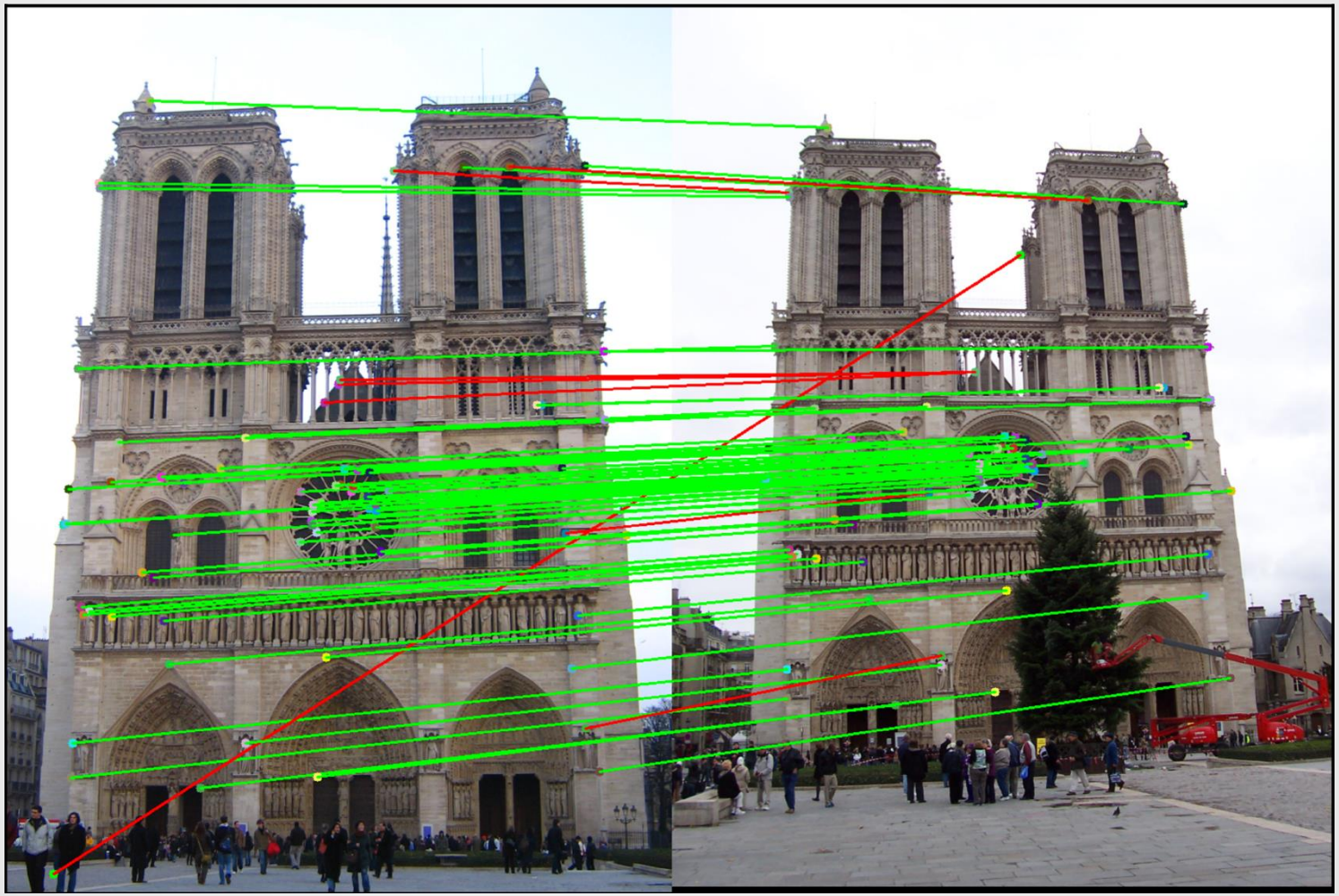
"Valve Reveals Timeline of Vive Prototypes", 2015

# Infrastructure-Mounted Markers

# Installing Artificial Markers



# Naturally Occurring Features



From https://www.cc.gatech.edu/~hays/compvision/proj2/

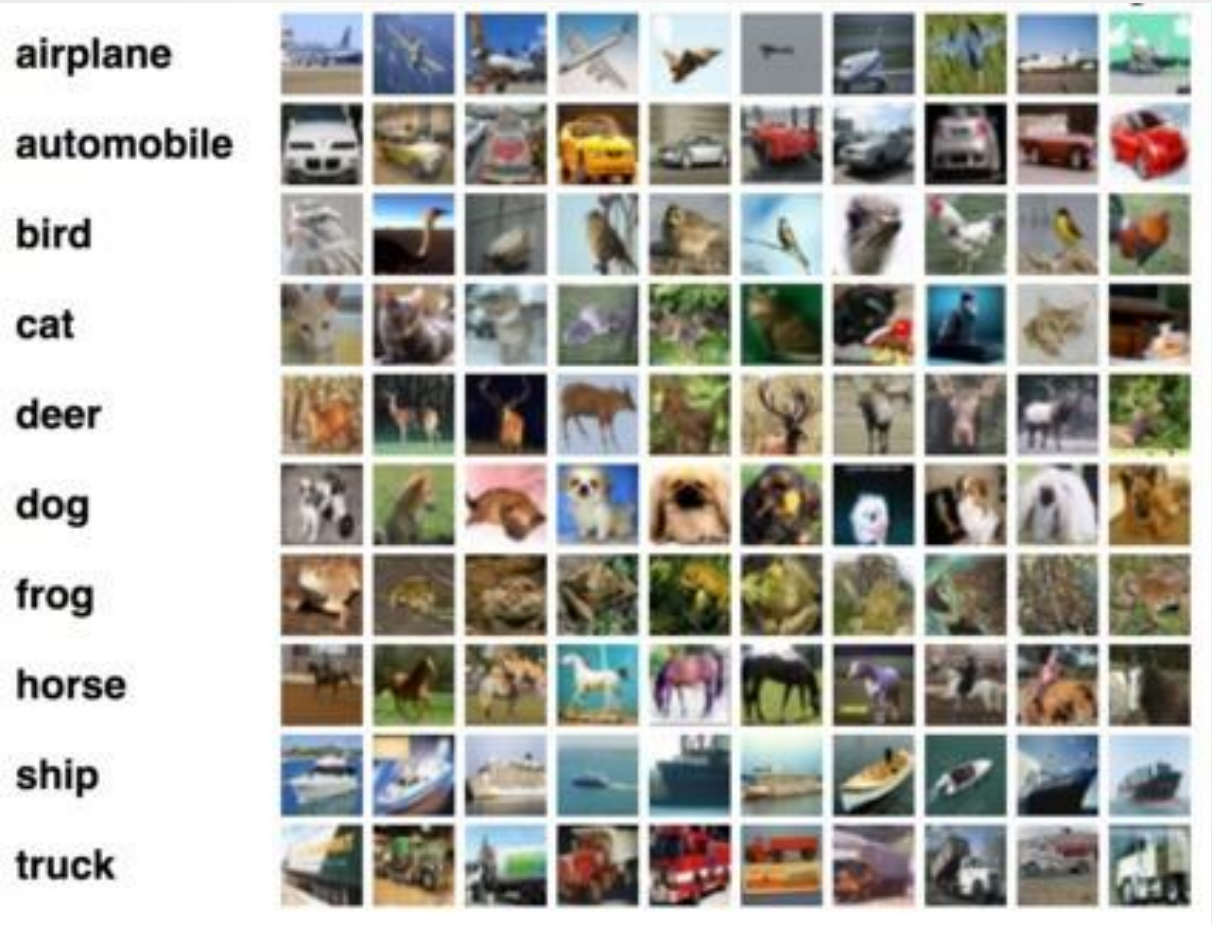
# Distinct Landmarks

# Non-Distinct Landmarks

# Characteristics of Landmarks

- ***They must be detectable by the sensing systems and feature detection algorithms***
- ***They must be distinct from one another***
- **When observing them, spatial information can be obtained**

# Image Labelling Detection, no Localization

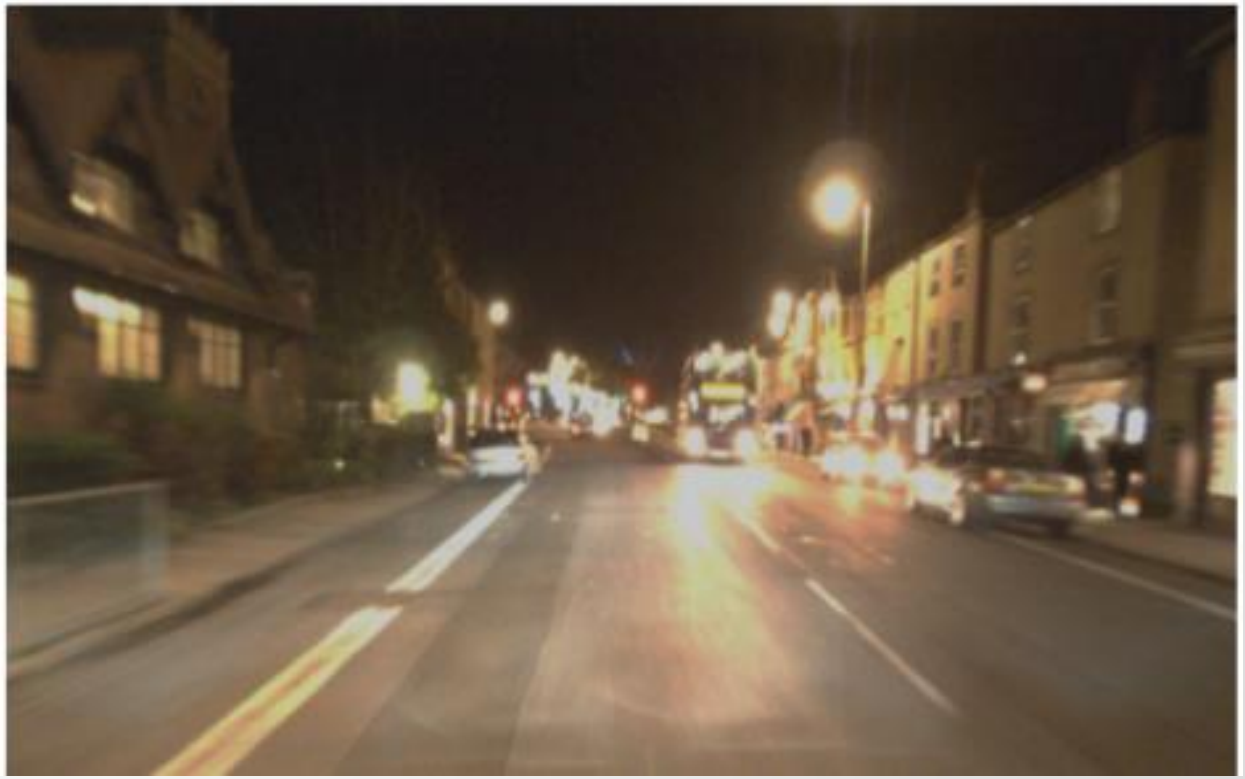



From [Achieving 90% accuracy in Object Recognition Task on CIFAR-10 Dataset with Keras: Convolutional Neural Networks]

# Robust Feature Recognition

"LoST? Appearance-Invariant Place Recognition for Opposite Viewpoints using Visual Semantics", Garg et al., RSS 2018

# Robust Semantic Matching



"LoST? Appearance-Invariant Place Recognition for Opposite Viewpoints using Visual Semantics", Garg et al., RSS 2018

# Robust Semantic Matching

"LoST? Appearance-Invariant Place Recognition for Opposite Viewpoints using Visual Semantics", Garg et al., RSS 2018

# Notation

- *Localization, Mapping, Odometry, SLAM*
- *Sparse Feature Point Maps*
- Notation

# Mathematical Description

- We have to define notation for:
  - Platform state
  - Platform process model
  - Landmark state
  - Platform / landmark observation model

# Platform State

- This is the state that the robot is in on a timestep-by-timestep basis
- For example, for a driven car with position and heading, the state might be

$$\mathbf{x}_k = \begin{bmatrix} x_k & y_k & \theta_k \end{bmatrix}^\top$$

# Platform Process Model

- **The platform state evolves according to the process model**

$$\mathbf{x}_{k+1} = \mathbf{f}\left[\mathbf{x}_k, \mathbf{u}_k, \mathbf{v}_k\right]$$

# Map State

- The map is a set of $N$ landmarks or features

$$\mathbf{m} = \{\mathbf{m}^1, \mathbf{m}^2, \cdots, \mathbf{m}^N\}$$

- Each landmark maintains its own state, such as its position

$$\mathbf{m}^i = \begin{bmatrix} x^i & y^i \end{bmatrix}^\top$$

# Map State

- We make two innocent-looking assumptions:

  1. All landmarks are static and don’t change over time

  2. All the landmarks have a unique identifier associated with them

# Landmarks Are Static

# Identifying Landmarks

- Each landmark has a unique identifier associated with it
- This can become very complicated when you don’t know how many landmarks are out there, there are multiple robots, etc.
- We are going to use a simple integer counter which increases by 1 each time

# Describing Sensor Observations

- At time $k$, the robot observes a set of $M_k$ landmarks,

$$\mathbf{Z}_k = \left\{ \mathbf{z}_k^1, \ \mathbf{z}_k^2, \ \ldots, \ \mathbf{z}_k^{M_k} \right\}$$

- Each observation comes from a landmark

# Sensor Observations

$$\mathbf{Z}_k = \left\{ \mathbf{z}_k^1, \ \mathbf{z}_k^2, \ \ldots, \ \mathbf{z}_k^{M_k} \right\}$$

# Describing Sensor Observations

- At time $k$, the robot observes a set of $M_k$ landmarks,

$$\mathbf{Z}_k = \left\{ \mathbf{z}_k^1, \ \mathbf{z}_k^2, \ \ldots, \ \mathbf{z}_k^{M_k} \right\}$$

- Each observation comes from a landmark
- The mapping index is

$$I_k = \left\{ i_k^1, \ i_k^2, \ \ldots, \ i_k^{M_k} \right\}$$

# Sensor Observations

$$\mathbf{Z}_k = \left\{ \mathbf{z}_k^1,\ \mathbf{z}_k^2,\ \ldots,\ \mathbf{z}_k^{M_k} \right\}$$

$$\boldsymbol{I}_k = \left\{ \boldsymbol{i}_k^1,\ \boldsymbol{i}_k^2,\ \ldots,\ \boldsymbol{i}_k^{M_k} \right\}$$

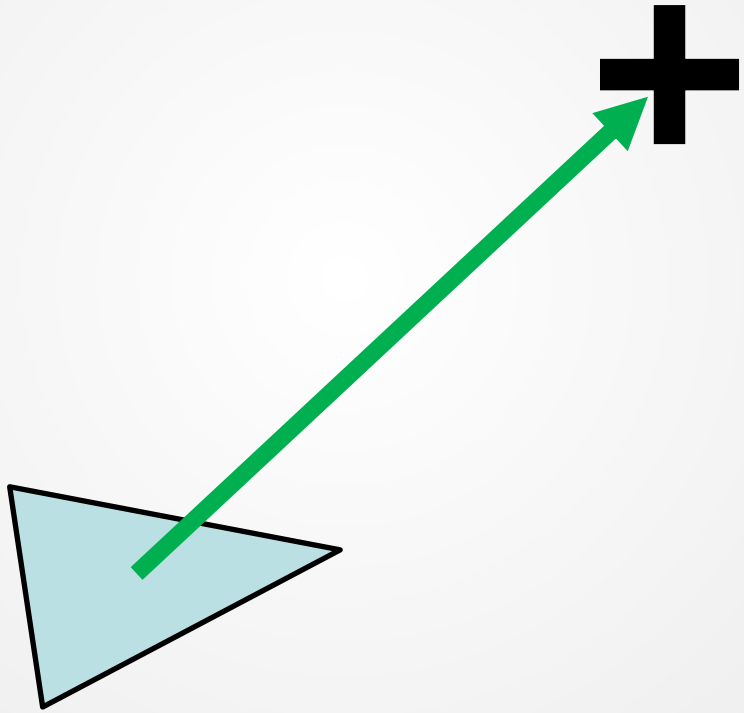
# Observation Models

- The observation model for the *j*th landmark

$$\mathbf{z}_k^j = \mathbf{h}\left[\mathbf{x}_k, \mathbf{m}^{i_j}, \mathbf{w}_k^j\right]$$

- This is a function of the *relative transformation* between the platform and the landmark
- This has a *profound* effect on SLAM

# Observation Models



# Probabilistic Equation for SLAM

- **The expression for the probabilistic formulation is**

$$f\left(\mathbf{x}_k, \mathbf{m} \middle| \mathbf{Z}_{0:k}, \mathbf{U}_{0:k}, \mathbf{x}_0\right)$$

**Current pose of the platform**

**Map**

$$\mathbf{m} = \left\{\mathbf{m}^1, \cdots, \mathbf{m}^{N_k}\right\}$$

**Set of all observations**

$$\mathbf{Z}_{0:k} = \{\mathbf{Z}_0, \ldots, \mathbf{Z}_k\}$$

**Set of all control inputs**

$$\mathbf{U}_{0:k} = \{\mathbf{u}_0, \ldots, \mathbf{u}_k\}$$

**Initial conditions**

# So… How Do We Implement SLAM?

- We have the problem that we want to estimate the joint distribution of the pose of the platform and the landmarks in one go
- This is an estimation algorithm which takes in a set of observations and control inputs to estimate a state vector
- We have already seen a solution – *the Kalman filter*